

EEE 304 (July 2025)

Digital Electronics Laboratory

Final Project Report

Section: A1 Group: 02

ECG Denoising Using an FPGA-Based FIR Low-Pass Filter with Comparison of Different Filter Orders

Course Instructors:

1. Dr. Hamidur Rahman, Professor, Department of EEE, BUET
2. Tanvir Hossain, Lecturer, Department of EEE, BUET

Signature of Instructor: _____

Academic Honesty Statement:

IMPORTANT! Please carefully read and sign the Academic Honesty Statement, below. Type the student ID and name, and put your signature. You will not receive credit for this project experiment unless this statement is signed in the presence of your lab instructor.

"In signing this statement, We hereby certify that the work on this project is our own and that we have not copied the work of any other students (past or present), and cited all relevant sources while completing this project. We understand that if we fail to honor this agreement, We will each receive a score of ZERO for this project and be subject to failure of this course."

Signature: _____

Full Name: Rejoun Rahi Rad

Student ID: 2106003

Signature: _____

Full Name: Abrar Fahim

Student ID: 2106005

Signature: _____

Full Name: Md. Yeasir Alif

Student ID: 2106011

Signature: _____

Full Name: Al Muktadir Khan

Student ID: 2106012

Table of Contents

1	Abstract.....	1
2	Introduction	1
3	Design	2
3.1	Problem Formulation	2
3.1.1	Identification of Scope	2
3.1.2	Literature Review.....	2
3.1.3	Formulation of Problem.....	3
3.1.4	Analysis	3
3.2	Design Method.....	3
3.3	Circuit Diagram	5
3.4	Full Source Code of Firmware.....	6
4	Implementation	10
4.1	Overview of System Workflow	10
4.2	ECG Signal Generation and Noise Modeling.....	10
4.3	FIR Filter Design	12
4.4	FPGA Implementation.....	14
4.4.1	FIR Filter Module	14
4.4.2	UART Receiver Module.....	19
4.4.3	UART Transmitter Module.....	21
4.4.4	Top-Level Module	25
4.5	UART Communication and Python Interface.....	29
5	Design Analysis and Evaluation	30
5.1	Novelty.....	30
5.2	Design Considerations	30
5.2.1	Considerations to public health and safety	31
5.2.2	Considerations to environment	31
5.2.3	Considerations to cultural and societal needs	31
5.3	Investigations	32
5.3.1	Design of Experiment	32
5.3.2	Data Collection	32
5.3.3	Results and Analysis	33
5.3.4	Interpretation and Conclusions on Data.....	34

5.4	Limitations of Tools.....	35
5.5	Impact Assessment.....	36
5.5.1	Assessment of Societal and Cultural Issues	36
5.5.2	Assessment of Health and Safety Issues	36
5.5.3	Assessment of Legal Issues	36
5.6	Sustainability Evaluation	37
5.7	Ethical Issues	38
6	Reflection on Individual and Team work	39
6.1	Individual Contribution of Each Member	39
6.2	Log Book of Project Implementation	40
7	Communication to External Stakeholders	40
7.1	Executive Summary	40
7.2	User Manual.....	41
7.3	Github Link.....	42
7.4	YouTube Link.....	42
8	Future Work.....	42
9	References	43

1 Abstract

Electrocardiogram (ECG) signals are widely used for monitoring and diagnosing cardiac conditions. However, these signals are often contaminated by high-frequency noise, which can obscure important waveform features and reduce diagnostic accuracy. This project presents the design and implementation of an ECG signal denoising system using a Finite Impulse Response (FIR) low-pass filter on an FPGA platform.

The FIR filter is designed using the Kaiser window method with specified parameters, including a sampling frequency of 360 Hz, a cutoff frequency of 40 Hz, and an optimized filter order. The filter coefficients are generated using MATLAB and converted into fixed-point Q15 format to enable efficient hardware implementation. The system is implemented in Verilog on the Nexys A7 Artix-100T FPGA board, incorporating modular components such as the FIR filter, UART communication modules, and a control unit.

To facilitate real-time data processing, ECG samples are transmitted from a host computer to the FPGA via a UART interface using a Python-based application. The FPGA performs filtering operations and returns the processed signal to the host system, where it is stored and analyzed. Both software-level and hardware-level simulations are conducted to validate the correctness of the design before deployment.

The results demonstrate effective noise reduction while preserving the essential characteristics of the ECG waveform. The proposed system provides a reliable and efficient solution for real-time ECG signal denoising, highlighting the suitability of FPGA-based implementations for biomedical signal processing applications.

2 Introduction

Electrocardiography is one of the most widely used non-invasive techniques for monitoring the electrical activity of the heart. The electrocardiogram (ECG) provides critical information about cardiac rhythm, conduction abnormalities, and overall heart health. Accurate interpretation of ECG signals is essential for early diagnosis and treatment of cardiovascular diseases.

However, ECG signals are often corrupted by various types of noise during acquisition. Common sources of interference include muscle activity, power line disturbances, and motion artifacts. These unwanted components can distort the signal and obscure important features such as the P-wave, QRS complex, and T-wave, thereby reducing diagnostic reliability. Consequently, effective noise reduction techniques are necessary to enhance signal quality without compromising the integrity of the underlying waveform.

Digital signal processing techniques play a crucial role in addressing this challenge. Among these, Finite Impulse Response (FIR) filters are widely preferred due to their inherent stability, linear phase characteristics, and ease of implementation. A properly designed low-pass FIR filter can effectively suppress high-frequency noise while preserving the essential features of the ECG signal.

With the increasing demand for real-time processing in biomedical applications, hardware-based implementations have gained significant importance. Field Programmable Gate Arrays (FPGAs) provide a flexible and efficient platform for implementing digital signal processing algorithms. Their

parallel processing capability, high speed, and reconfigurability make them well-suited for real-time ECG signal filtering.

In this project, an ECG denoising system is designed and implemented using a FIR low-pass filter on an FPGA platform. The filter is designed using MATLAB and implemented in Verilog hardware description language. A UART-based communication interface is used to transfer ECG data between a host computer and the FPGA, enabling real-time processing and validation of the system.

3 Design

3.1 Problem Formulation

3.1.1 Identification of Scope

The scope of this project is to design and implement a hardware-based system for denoising electrocardiogram (ECG) signals using a finite impulse response (FIR) filter on an FPGA platform. The system focuses on removing high-frequency noise components, particularly muscle noise, while preserving the essential characteristics of the ECG waveform.

The project encompasses the generation of a synthetic ECG signal, the addition of noise to simulate real-world conditions, and the design of a digital filter to recover the clean signal. The filtering operation is implemented on a Nexys A7 Artix-100T FPGA board, enabling real-time processing of input data.

The system also includes a communication interface using UART, through which ECG samples are transmitted from a host computer to the FPGA and the filtered results are returned. The scope is limited to digital signal processing and FPGA-based implementation, and does not include hardware sensor acquisition or analog front-end circuitry.

3.1.2 Literature Review

ECG signal denoising is a widely studied problem in biomedical signal processing. Various techniques have been proposed for noise removal, including digital filtering, wavelet-based methods, and adaptive filtering approaches.

Among these, FIR filters are commonly used due to their inherent stability and linear phase characteristics, which ensure that the morphology of the ECG signal is preserved during filtering. Linear phase response is particularly important for maintaining the timing and shape of critical features such as the QRS complex.

Previous implementations of ECG denoising have been carried out using both software-based platforms and hardware-based systems. FPGA-based implementations have gained attention due to their ability to perform high-speed parallel processing with low latency, making them suitable for real-time biomedical applications.

3.1.3 Formulation of Problem

ECG signals acquired in practical environments are often contaminated with various types of noise, including muscle artifacts, power line interference, and baseline wander. These noise components can distort the signal and make accurate diagnosis difficult.

The problem addressed in this project is to design a system capable of effectively removing high-frequency noise from an ECG signal while preserving its essential features. Specifically, the system must:

- Accept a noisy ECG signal as input
- Filter out unwanted high-frequency components
- Maintain the integrity of clinically important waveform features
- Operate in real-time using FPGA hardware

The challenge lies in achieving an optimal balance between noise reduction and signal preservation while considering hardware constraints such as limited precision and computational resources.

3.1.4 Analysis

To address the problem, a low-pass FIR filter is selected as the primary denoising technique. Since most ECG signal components lie below 40 Hz, a cutoff frequency of 40 Hz is chosen to allow the desired signal to pass while attenuating higher-frequency noise.

The use of FIR filtering ensures stability and linear phase response, which prevents distortion of the ECG waveform. The filter is designed using the Kaiser window method, allowing control over the trade-off between transition width and stopband attenuation.

The implementation on FPGA requires the use of fixed-point arithmetic, specifically Q15 format, to ensure efficient hardware utilization. This introduces quantization effects, which are considered during the design process.

Additionally, UART communication is analyzed as the data transfer mechanism between the host computer and FPGA. While UART provides a simple and reliable interface, it introduces limitations in data throughput, which must be considered in system performance.

Overall, the analysis leads to a design that balances signal processing requirements with hardware constraints, resulting in an efficient and practical ECG denoising system.

3.2 Design Method

The design of the proposed ECG denoising system follows a structured approach that integrates signal modeling, digital filter design, and hardware implementation to achieve effective noise suppression while preserving the essential characteristics of the ECG waveform.

The process begins with the generation of a synthetic ECG signal using a mathematical model. The waveform is constructed by combining multiple Gaussian functions, each representing a specific component of the cardiac cycle such as the P-wave, QRS complex, and T-wave. This approach

allows controlled generation of ECG signals with realistic morphology. To simulate practical conditions, high-frequency noise is added to the signal, enabling evaluation of the system under conditions similar to real-world ECG acquisition.

The denoising process is based on digital signal processing techniques. A low-pass finite impulse response (FIR) filter is selected due to its linear phase response and inherent stability, which are essential for preserving the temporal features of ECG signals. The filtering operation is based on the discrete convolution process, where the output signal is obtained as a weighted sum of current and past input samples:

$$y[n] = \sum_{k=0}^{N-1} h[k] \cdot x[n - k]$$

where $x[n]$ represents the input signal, $h[k]$ denotes the filter coefficients, and $y[n]$ is the filtered output signal.

The design of the FIR filter is carried out using the Kaiser window method, which provides flexibility in controlling the trade-off between transition bandwidth and stopband attenuation. Analytical relationships are used to estimate the filter parameters, including the filter order and window shape factor. These theoretical values are subsequently adjusted to achieve a practical balance between filtering performance and hardware resource constraints.

To ensure compatibility with hardware implementation, the filter coefficients are converted from floating-point representation to fixed-point format. A Q15 representation is adopted, where each coefficient is scaled according to:

$$h_{Q15} = \text{round}(h \times 2^{15})$$

This scaling converts fractional values into 16-bit signed integers, enabling efficient arithmetic operations on the FPGA while maintaining sufficient numerical precision.

The hardware implementation is performed on the Nexys A7 Artix-100T using Verilog HDL. The system is organized into modular components, including the FIR filter module for signal processing, UART receiver and transmitter modules for communication, and a top-level control unit. A finite state machine is used to coordinate data reception, filtering, and transmission, ensuring proper synchronization between system components.

Communication between the FPGA and the host computer is established through a UART interface. A Python-based program is used to transmit ECG samples to the FPGA and receive the filtered output. This setup enables real-time testing and verification of the system performance.

Overall, the design method combines mathematical modeling, digital signal processing principles, and hardware-oriented implementation techniques to develop an efficient and reliable ECG denoising system suitable for real-time applications.

3.3 Circuit Diagram

The proposed system is implemented using a digital hardware architecture on FPGA rather than a conventional analog circuit. Therefore, instead of a traditional circuit schematic, the design is represented using a hardware-level architectural diagram.

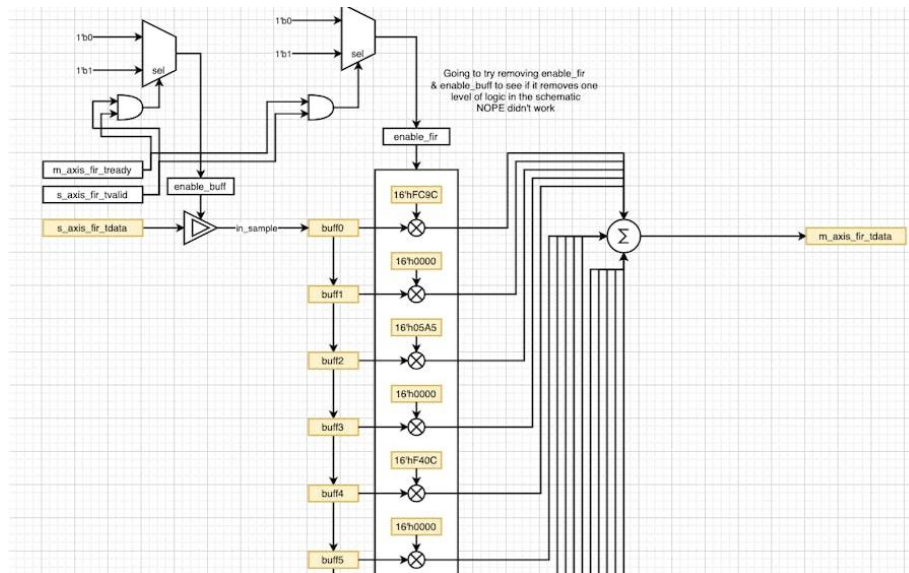


Figure 1: Hardware Architecture of FIR Filter Module

The diagram illustrates the internal structure of the FIR filter module implemented in Verilog. The design consists of a series of delay elements (shift registers) that store the current and previous input samples. Each stored sample is multiplied by a corresponding filter coefficient.

The multiplication results are then summed using an accumulator to produce the filtered output. This operation follows the discrete convolution process used in FIR filtering.

Control signals such as valid, ready, and enable are used to synchronize data flow and ensure correct timing during processing.

3.4 Full Source Code of Firmware

```
// Verilog code
//Top module
`timescale 1ns / 1ps
module top (
    input wire clk,
    input wire reset_btn,
    input wire uart_rx_pin,
    output wire uart_tx_pin
);
    wire fir_reset_n = ~reset_btn;

    wire [7:0] rx_byte;
    wire rx_valid;

    uart_rx #(CLK_PER_BIT(868)) u_rx (
        .clk (clk),
        .reset (reset_btn),
        .rx (uart_rx_pin),
        .data (rx_byte),
        .valid (rx_valid)
    );

    reg [7:0] tx_byte;
    reg tx_start;
    wire tx_busy;

    uart_tx #(CLK_PER_BIT(868)) u_tx (
        .clk (clk),
        .reset (reset_btn),
        .data (tx_byte),
        .start (tx_start),
        .tx (uart_tx_pin),
        .busy (tx_busy)
    );

    reg signed [15:0] fir_in_data;
    reg fir_in_valid;
    wire signed [31:0] fir_out_data;
    wire fir_out_valid;
    wire fir_in_ready;

    FIR fir_inst (
        .clk (clk),
        .reset (fir_reset_n),
        .s_axis_fir_tdata (fir_in_data),
        .s_axis_fir_tvalid (fir_in_valid),
        .s_axis_fir_tlast (1'b0),
        .s_axis_fir_tready (fir_in_ready),
        .m_axis_fir_tdata (fir_out_data),
        .m_axis_fir_tvalid (fir_out_valid),
        .m_axis_fir_tlast (),
        .m_axis_fir_tkeep (),
        .m_axis_fir_tready (1'b1)
    );

    localparam S_RECV_B0 = 4'd0,
               S_RECV_B1 = 4'd1,
               S_FILTER = 4'd2,
               S_SEND_B0 = 4'd3,
               S_WAIT_B0 = 4'd4,
               S_SEND_B1 = 4'd5,
               S_WAIT_B1 = 4'd6,
               S_SEND_B2 = 4'd7,
               S_WAIT_B2 = 4'd8,
               S_SEND_B3 = 4'd9,
               S_WAIT_B3 = 4'd10;

    reg [3:0] state;
    reg [7:0] byte_lo;
    reg signed [31:0] result;
    reg fir_sent;
    reg tx_was_busy;

    always @(posedge clk) begin
        tx_start <= 1'b0;
        fir_in_valid <= 1'b0;
    end
    if (reset_btn) begin
        state <= S_RECV_B0;
        fir_sent <= 1'b0;
        tx_was_busy <= 1'b0;
    end

    fir_in_data <= 16'sd0;
    end else begin
        case (state)
            S_RECV_B0: begin
                if (rx_valid) begin
                    byte_lo <= rx_byte;
                    state <= S_RECV_B1;
                end
            end
            S_RECV_B1: begin
                if (rx_valid) begin
                    fir_in_data <=
                        $signed({rx_byte, byte_lo});
                    fir_sent <= 1'b0;
                    state <= S_FILTER;
                end
            end
            S_FILTER: begin
                if (!fir_sent) begin
                    fir_in_valid <= 1'b1;
                    fir_sent <= 1'b1;
                end
                if (fir_out_valid) begin
                    result <= fir_out_data;
                    state <= S_SEND_B0;
                end
            end
            S_SEND_B0: begin
                tx_byte <= result[7:0];
                tx_start <= 1'b1;
                tx_was_busy <= 1'b0;
                state <= S_WAIT_B0;
            end
            S_WAIT_B0: begin
                if (tx_busy)
                    tx_was_busy <= 1'b1;
                if (tx_was_busy && !tx_busy) state
                    <= S_SEND_B1;
            end
            S_SEND_B1: begin
                tx_byte <= result[15:8];
                tx_start <= 1'b1;
                tx_was_busy <= 1'b0;
                state <= S_WAIT_B1;
            end
            S_WAIT_B1: begin
                if (tx_busy)
                    tx_was_busy <= 1'b1;
                if (tx_was_busy && !tx_busy) state
                    <= S_SEND_B2;
            end
            S_SEND_B2: begin
                tx_byte <= result[23:16];
                tx_start <= 1'b1;
                tx_was_busy <= 1'b0;
                state <= S_WAIT_B2;
            end
            S_WAIT_B2: begin
                if (tx_busy)
                    tx_was_busy <= 1'b1;
                if (tx_was_busy && !tx_busy) state
                    <= S_SEND_B3;
            end
            S_SEND_B3: begin
                tx_byte <= result[31:24];
                tx_start <= 1'b1;
                tx_was_busy <= 1'b0;
                state <= S_WAIT_B3;
            end
            S_WAIT_B3: begin
                if (tx_busy)
                    tx_was_busy <= 1'b1;
                if (tx_was_busy && !tx_busy) state
                    <= S_RECV_B0;
            end
            default: state <= S_RECV_B0;
        endcase
    end
endmodule
```

```

// uart_rx module

`timescale 1ns / 1ps
module uart_rx (
    input wire      clk,
    input wire      reset,
    input wire      rx,
    output reg [7:0] data,
    output reg      valid
);
    parameter CLK_PER_BIT = 868;

    localparam IDLE = 2'd0,
               START = 2'd1,
               DATA = 2'd2,
               STOP = 2'd3;

    reg [9:0] clk_cnt;
    reg [2:0] bit_idx;
    reg [1:0] state;
    reg [7:0] rx_data;

    always @(posedge clk) begin
        valid <= 1'b0;
        if (reset) begin
            state <= IDLE;
            clk_cnt <= 0;
            bit_idx <= 0;
        end else begin
            case (state)
                IDLE: begin
                    if (rx == 1'b0) begin
                        clk_cnt <= 1;
                        state <= START;
                    end
                end
                START: begin
                    if (clk_cnt == CLK_PER_BIT/2) begin
                        if (rx == 1'b0) begin
                            clk_cnt <= 1;
                            bit_idx <= 0;
                            state <= DATA;
                        end else
                            state <= IDLE;
                    end else
                        clk_cnt <= clk_cnt + 1;
                end
                DATA: begin
                    if (clk_cnt == CLK_PER_BIT) begin
                        rx_data[bit_idx] <= rx;
                        clk_cnt <= 1;
                        if (bit_idx == 3'd7)
                            state <= STOP;
                        else
                            bit_idx <= bit_idx + 1;
                    end else
                        clk_cnt <= clk_cnt + 1;
                end
                STOP: begin
                    if (clk_cnt == CLK_PER_BIT) begin
                        valid <= 1'b1;
                        data <= rx_data;
                        state <= IDLE;
                        clk_cnt <= 0;
                    end else
                        clk_cnt <= clk_cnt + 1;
                end
            endcase
        end
    end
endmodule

```

```

// uart_tx module

`timescale 1ns / 1ps
module uart_tx (
    input wire      clk,
    input wire      reset,
    input wire [7:0] data,
    input wire      start,
    output reg      tx,
    output reg      busy
);
    parameter CLK_PER_BIT = 868;

    localparam IDLE = 2'd0,
               START = 2'd1,
               DATA = 2'd2,
               STOP = 2'd3;

    reg [9:0] clk_cnt;
    reg [2:0] bit_idx;
    reg [1:0] state;
    reg [7:0] tx_data;

    always @(posedge clk) begin
        if (reset) begin
            state <= IDLE;
            tx <= 1'b1;
            busy <= 1'b0;
            clk_cnt <= 0;
            bit_idx <= 0;
        end else begin
            case (state)
                IDLE: begin
                    tx <= 1'b1;
                    busy <= 1'b0;
                    if (start) begin
                        tx_data <= data;
                        busy <= 1'b1;
                        clk_cnt <= 1;
                        state <= START;
                    end
                end
                START: begin
                    tx <= 1'b0;
                    if (clk_cnt == CLK_PER_BIT) begin
                        clk_cnt <= 1;
                        bit_idx <= 0;
                        state <= DATA;
                    end else
                        clk_cnt <= clk_cnt + 1;
                end
                DATA: begin
                    tx <= tx_data[bit_idx];
                    if (clk_cnt == CLK_PER_BIT) begin
                        clk_cnt <= 1;
                        if (bit_idx == 3'd7)
                            state <= STOP;
                        else
                            bit_idx <= bit_idx + 1;
                    end else
                        clk_cnt <= clk_cnt + 1;
                end
                STOP: begin
                    tx <= 1'b1;
                    if (clk_cnt == CLK_PER_BIT) begin
                        state <= IDLE;
                        busy <= 1'b0;
                        clk_cnt <= 0;
                    end else
                        clk_cnt <= clk_cnt + 1;
                end
            endcase
        end
    end
endmodule

```

<pre>// FIR module `timescale 1ns / 1ps module FIR #(parameter NUM_TAPS = 17)(input wire clk, input wire reset, /* AXI-Stream input */ input wire signed [15:0] s_axis_fir_tdata, input wire s_axis_fir_tvalid, input wire s_axis_fir_tlast, output wire s_axis_fir_tready, /* AXI-Stream output */ output reg signed [31:0] m_axis_fir_tdata, output reg m_axis_fir_tvalid, output reg m_axis_fir_tlast, output reg [3:0] m_axis_fir_tkeep, input wire m_axis_fir_tready); assign s_axis_fir_tready = 1'b1; wire in_xfer = s_axis_fir_tvalid && s_axis_fir_tready; wire out_xfer = m_axis_fir_tvalid && m_axis_fir_tready; reg signed [15:0] x[0:NUM_TAPS-1]; integer i; always @(posedge clk) begin if (!reset) begin for (i = 0; i < NUM_TAPS; i = i + 1) x[i] <= 16'sd0; end else if (in_xfer) begin x[0] <= s_axis_fir_tdata; for (i = 1; i < NUM_TAPS; i = i + 1) x[i] <= x[i-1]; end end reg signed [15:0] h[0:NUM_TAPS-1]; initial begin h[0] = 16'hFFEF; h[1] = 16'hFF89; h[2] = 16'hFEE9; h[3] = 16'hFF13; h[4] = 16'h01C5; h[5] = 16'h0823; h[6] = 16'h111E; h[7] = 16'h1940; h[8] = 16'h1C8C; h[9] = 16'h1940; h[10] = 16'h111E; h[11] = 16'h0823; h[12] = 16'h01C5; h[13] = 16'hFF13; h[14] = 16'hFEE9; h[15] = 16'hFF89; h[16] = 16'hFFEF; end reg signed [35:0] acc; integer j; always @(*) begin acc = 36'sd0; for (j = 0; j < NUM_TAPS; j = j + 1) acc = acc + x[j] * h[j]; end always @(posedge clk) begin if (!reset) begin m_axis_fir_tvalid <= 1'b0; m_axis_fir_tdata <= 32'sd0; m_axis_fir_tlast <= 1'b0; m_axis_fir_tkeep <= 4'hF; end end end</pre>	<pre>always @(posedge clk) begin if (!reset) begin m_axis_fir_tvalid <= 1'b0; m_axis_fir_tdata <= 32'sd0; m_axis_fir_tlast <= 1'b0; m_axis_fir_tkeep <= 4'hF; end else begin if (in_xfer && (!m_axis_fir_tvalid m_axis_fir_tready)) begin m_axis_fir_tdata <= acc >> 15; m_axis_fir_tvalid <= 1'b1; m_axis_fir_tlast <= s_axis_fir_tlast; m_axis_fir_tkeep <= 4'hF; end else if (out_xfer) begin m_axis_fir_tvalid <= 1'b0; end end end endmodule // Python code // ECG generation import numpy as np fs = 360 duration = 5 heart_rate = 60 rr_interval = 60 / heart_rate muscle_noise_amp = 0.05 t = np.linspace(0, duration, fs * duration, endpoint=False) def gaussian(t, mu, sigma, A): return A * np.exp(-0.5 * ((t - mu) / sigma) ** 2) ecg = np.zeros_like(t) for beat in np.arange(0, duration, rr_interval): ecg += gaussian(t, beat + 0.10, 0.025, 0.10) ecg += gaussian(t, beat + 0.20, 0.010, -0.15) ecg += gaussian(t, beat + 0.22, 0.012, 1.00) ecg += gaussian(t, beat + 0.24, 0.010, -0.25) ecg += gaussian(t, beat + 0.35, 0.040, 0.30) ecg = ecg / np.max(np.abs(ecg)) np.random.seed(0) muscle_noise = muscle_noise_amp * np.random.randn(len(t)) ecg_muscle = ecg + muscle_noise ecg_muscle = ecg_muscle / np.max(np.abs(ecg_muscle)) ecg_q15 = np.int16(ecg_muscle * 32767) np.savetxt("ecg_muscle_input.txt", ecg_q15, fmt="%d") print("FPGA input file generated:") print(" -> ecg_muscle_input.txt") print("Format: Q15 signed integers")</pre>
---	---

<pre>// Python code // Sending and receiving data from FPGA import serial import struct import time import sys PORT = 'COM4' BAUD = 115200 INPUT_FILE = 'ecg.txt' OUTPUT_FILE = 'ecg_filtered.txt' def main(): with open(INPUT_FILE, 'r') as f: samples = [int(line.strip()) for line in f if line.strip()] print("Loaded {} ECG samples from {}".format(len(samples), INPUT_FILE)) samples_clamped = [max(-32768, min(32767, s)) for s in samples] filtered = [] try: ser = serial.Serial(PORT, BAUD, timeout=10) except serial.SerialException as e: print("ERROR: Cannot open {}".format(PORT)) print(str(e)) print("Check that the Nexys A7 is connected and the COM port is correct.") sys.exit(1) print("Connected to {} at {} baud.".format(PORT, BAUD)) time.sleep(1) ser.reset_input_buffer() ser.reset_output_buffer() for i, sample in enumerate(samples_clamped): tx_bytes = struct.pack('<h', sample) ser.write(tx_bytes) rx_bytes = ser.read(4) if len(rx_bytes) < 4: print("\nTIMEOUT at sample {}. Only got {} bytes back.".format(i, len(rx_bytes))) print("Check that the FPGA is programmed and the board is on.") break value = struct.unpack('<i', rx_bytes)[0] filtered.append(value) if (i + 1) % 100 == 0 or (i + 1) == len(samples): print("Progress: {}/{} samples processed".format(i + 1, len(samples)))</pre>	<pre>ser.close() print("\nDone. {} samples received.".format(len(filtered))) with open(OUTPUT_FILE, 'w') as f: for v in filtered: f.write("{}\n".format(v)) print("Filtered signal saved to {}".format(OUTPUT_FILE)) if __name__ == '__main__': main()</pre>
--	--

Table: Source Code for the main program

4 Implementation

4.1 Overview of System Workflow

The implementation of the proposed ECG denoising system is carried out through a combination of signal processing, hardware design, and software–hardware interfacing. The overall workflow begins with the generation of a synthetic ECG signal, followed by the addition of noise to simulate real-world conditions. A digital FIR low-pass filter is then designed using MATLAB’s filter design tool based on specified parameters, including sampling frequency, cutoff frequency, filter order, and Kaiser window beta value.

The designed filter coefficients are extracted and converted into fixed-point Q15 hexadecimal format to ensure compatibility with FPGA-based processing. These coefficients are then implemented in a Verilog-based FIR filter module within the FPGA environment using Vivado.

To facilitate communication between the computer and the FPGA board, a Python-based serial interface is developed. The noisy ECG samples are transmitted from the computer to the FPGA via UART communication. The FPGA processes the incoming data in real time using the FIR filter and sends the filtered output back to the computer through the same UART interface. The received data is then stored and analyzed using Python, allowing comparison between the noisy and filtered ECG signals.

This workflow enables a complete end-to-end implementation of ECG signal denoising, integrating software-based signal generation, hardware-based real-time filtering, and post-processing for performance evaluation.

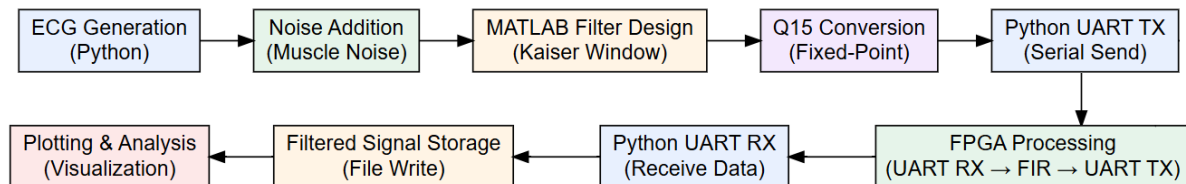


Figure 2: Overall System Workflow

4.2 ECG Signal Generation and Noise Modeling

In order to evaluate the performance of the proposed FIR filter, a synthetic ECG signal is generated using a mathematical model. The use of a generated signal allows complete control over signal characteristics such as amplitude, timing, and noise level, which is essential for systematic testing and validation of the filtering process. Unlike real-world ECG data, a synthetic signal ensures repeatability and consistency across experiments.

The ECG signal is generated using a combination of Gaussian functions to approximate the characteristic components of a cardiac cycle, namely the P-wave, QRS complex, and T-wave. Each component is modeled with appropriate amplitude, mean position, and standard deviation to closely resemble a realistic ECG waveform. The overall ECG signal is constructed by repeating this waveform periodically based on the heart rate.

Parameter Selection

The sampling frequency is chosen as 360 Hz, which is a commonly used standard in ECG signal processing. This sampling rate is sufficiently high to capture the essential frequency components of the ECG signal, particularly the QRS complex, without causing aliasing.

The duration of the signal is selected as 5 seconds, providing a sufficient number of cardiac cycles for analysis while keeping computational requirements manageable.

The heart rate is set to 60 beats per minute, corresponding to a normal resting heart rate. This results in an RR interval of 1 second, ensuring uniform spacing between successive cardiac cycles.

ECG Signal Modeling

Each ECG cycle is modeled using multiple Gaussian functions, where each Gaussian represents a specific feature of the ECG waveform. The use of Gaussian functions provides smooth transitions and allows precise control over waveform shape.

```
def gaussian(t, mu, sigma, A):  
    return A * np.exp(-0.5 * ((t - mu) / sigma) ** 2)
```

The ECG signal is generated by summing multiple Gaussian components for each heartbeat over the entire duration of the signal. The positions and amplitudes of these Gaussians are selected to approximate the morphology of a typical ECG waveform.

```
ecg = np.zeros_like(t)  
  
for beat in np.arange(0, duration, rr_interval):  
    ecg += gaussian(t, beat + 0.10, 0.025, 0.10)  
    ecg += gaussian(t, beat + 0.20, 0.010, -0.15)  
    ecg += gaussian(t, beat + 0.22, 0.012, 1.00)  
    ecg += gaussian(t, beat + 0.24, 0.010, -0.25)  
    ecg += gaussian(t, beat + 0.35, 0.040, 0.30)
```

After generation, the ECG signal is normalized to ensure that its amplitude lies within the range $[-1, 1]$. This normalization is necessary to prevent overflow during fixed-point conversion and to maintain consistency across different processing stages.

```
ecg = ecg / np.max(np.abs(ecg))
```

Noise Modeling

To simulate real-world conditions, noise is added to the generated ECG signal. In practical scenarios, ECG signals are often contaminated by muscle noise (electromyographic interference), which appears as high-frequency random variations.

In this project, muscle noise is modeled as additive white Gaussian noise with a small amplitude. The noise amplitude is selected as 0.05, which introduces noticeable distortion while still preserving the underlying ECG structure. This allows meaningful evaluation of the filter's performance.

```
np.random.seed(0)
muscle_noise = muscle_noise_amp * np.random.randn(len(t))

ecg_muscle = ecg + muscle_noise
ecg_muscle = ecg_muscle / np.max(np.abs(ecg_muscle))
```

Fixed-Point Conversion for FPGA

Since the FPGA processes data in fixed-point format, the generated noisy ECG signal is converted into Q15 representation. This involves scaling the signal to the range of 16-bit signed integers and casting it accordingly.

The Q15 format is chosen because it provides a good balance between numerical precision and hardware efficiency. It allows efficient implementation of arithmetic operations in FPGA while minimizing resource usage.

```
ecg_q15 = np.int16(ecg_muscle * 32767)
```

Data Export

Finally, the processed ECG samples are exported to a text file, which serves as the input dataset for the FPGA system. Each line in the file represents a single sample in Q15 format.

```
np.savetxt("ecg_muscle_input.txt", ecg_q15, fmt="%d")
```

The generated dataset provides a controlled test signal with realistic noise characteristics. This enables effective evaluation of the FIR filter implemented on the FPGA and ensures that the system can handle practical ECG denoising scenarios.

4.3 FIR Filter Design

The core component of our ECG denoising system is a low-pass FIR filter implemented on the FPGA. FIR (Finite Impulse Response) filters are chosen for their linear phase response, stability, and suitability for hardware implementation. The filter design is performed in MATLAB using the Filter Designer Tool and the Kaiser window method, allowing precise control over passband, stopband, and transition width.

Parameter Selection and Justification

- **Filter Order (16):** A moderate filter order balances resource usage on the FPGA and filter performance. Lower-order filters consume fewer multipliers and registers but may have insufficient stopband attenuation. Higher orders improve frequency selectivity but increase FPGA utilization and latency. Order 16 was selected as it provides adequate attenuation for typical ECG noise while keeping hardware requirements low.
- **Sampling Frequency ($F_s = 360$ Hz):** This matches the ECG signal generation stage and satisfies the Nyquist criterion for ECG frequency components.

- **Cutoff Frequency ($F_c = 40$ Hz):** Muscle noise and high-frequency artifacts often appear above 40 Hz. Setting $F_c = 40$ Hz allows preservation of the main ECG components (P, QRS, T waves) while removing high-frequency noise.
- **Kaiser Window Beta ($\beta = 5.65$):** The beta parameter controls the trade-off between main-lobe width and side-lobe attenuation. A β of 5.65 provides adequate stopband attenuation (~60 dB) without making the transition band excessively wide.

FIR Coefficient Generation

The filter is designed in MATLAB using the **Filter Designer App**, where the **low-pass FIR filter** is configured with the parameters mentioned above. Once designed, the filter coefficients are exported and converted into **Q15 fixed-point format** to match the FPGA's 16-bit signed input/output interface. This ensures numerical compatibility and prevents overflow.

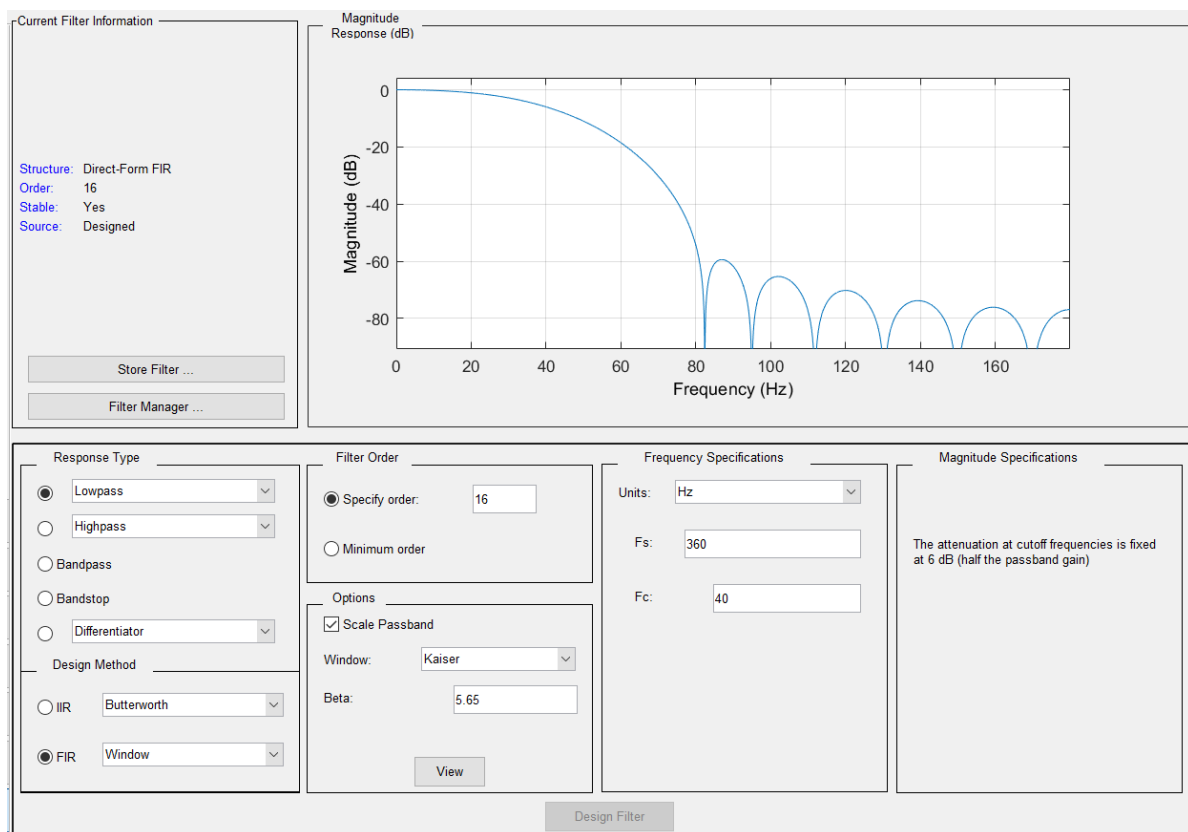


Figure 3: Screenshot of the MATLAB Filter Designer showing the low-pass FIR specifications

The resulting filter coefficients produce the expected low-pass frequency response, attenuating frequencies above 40 Hz while keeping passband distortion minimal.

Coefficient Extraction and Q15 Conversion

The filter coefficients obtained from MATLAB are represented in floating-point format. However, FPGA-based implementation requires fixed-point representation to ensure efficient hardware utilization. Therefore, the coefficients are converted into Q15 format, which is a 16-bit signed fixed-point representation. The Q15 format is chosen as a compromise between precision and hardware efficiency.

In Q15 format, one bit is used for the sign and the remaining 15 bits represent the fractional part. The conversion from floating-point to Q15 is performed by scaling each coefficient by 2^{15} and rounding to the nearest integer:

$$h_{Q15} = \text{round}(h \times 2^{15})$$

where h is the original floating-point coefficient and h_{Q15} is the fixed-point representation.

To ensure numerical stability and avoid overflow, the resulting values are constrained within the valid range of a 16-bit signed integer:

$$-32768 \leq h_{Q15} \leq 32767$$

After scaling and saturation, the coefficients are converted into two's complement hexadecimal format, which can be directly used in the Verilog implementation of the FIR filter.

This fixed-point representation significantly reduces computational complexity and enables efficient implementation of multiply-accumulate operations on the FPGA, while maintaining acceptable numerical accuracy for ECG signal processing.

4.4 FPGA Implementation

The FPGA implementation of the proposed ECG denoising system is carried out using Verilog HDL on the Nexys A7 Artix-100T board. The FPGA serves as the core processing unit, performing real-time filtering of incoming ECG samples. The design is modular in nature and consists of four primary components: the FIR filter module, UART receiver, UART transmitter, and a top-level control module.

The system operates by receiving ECG samples from a host computer via UART communication, processing the data using the FIR filter, and transmitting the filtered output back to the computer. This architecture enables efficient real-time signal processing using fixed-point arithmetic.

4.4.1 FIR Filter Module

The FIR module is the core signal processing unit of this project. It receives digitized ECG samples through an AXI-Stream interface, applies a set of pre-designed filter coefficients to attenuate unwanted noise and frequency components, and outputs the filtered result through a second AXI-Stream interface. The module is implemented in Verilog and is designed to operate on the Nexys A7-100T FPGA board running at a 100 MHz system clock.

The filter is a linear-phase lowpass FIR filter. The term "finite impulse response" refers to the

fundamental property that the filter's output depends only on a finite window of past input samples. Specifically, the current sample and the preceding $N-1$ samples, where N is the number of filter taps. Because no feedback from the output is used, FIR filters are inherently stable and, when designed with symmetric coefficients, introduce no phase distortion which means all frequency components in the passband are delayed by the same amount. This property is particularly important for ECG signals, where preserving the shape and timing of the QRS complex is critical for clinical interpretation.

Module Interface

The module is parameterized, meaning the filter order can be changed at instantiation time without modifying the internal logic. The parameter NUM_TAPS controls the number of filter taps (equivalent to filter order + 1). We are using 17 taps in accordance to our filter order specification.

```
module FIR #(
    parameter NUM_TAPS = 17
) (
    input wire clk,
    input wire reset,
    /* AXI-Stream input */
    input wire signed [15:0] s_axis_fir_tdata,
    input wire s_axis_fir_tvalid,
    input wire s_axis_fir_tlast,
    output wire s_axis_fir_tready,
    /* AXI-Stream output */
    output reg signed [31:0] m_axis_fir_tdata,
    output reg m_axis_fir_tvalid,
    output reg m_axis_fir_tlast,
    output reg [3:0] m_axis_fir_tkeep,
    input wire m_axis_fir_tready
);
```

The module follows the AXI-Stream on both its input and output sides. AXI-Stream is an industry-standard handshaking protocol in which a transfer occurs only when both the valid signal (asserted by the sender) and the ready signal (asserted by the receiver) are simultaneously HIGH. This protocol allows different components in the system to operate at their own pace and signal backpressure when they are not ready to accept data.

The input side uses the s_ prefix (slave, meaning this module receives data) and the output side uses the m_ prefix (master, meaning this module sends data). The input data 's_axis_fir_tdata' is a 16-bit signed integer representing one ECG sample in the range -32768 to $+32767$. The output 'm_axis_fir_tdata' is 32 bits wide to accommodate the larger dynamic range that results from the filtering computation.

Since the FIR filter is fully capable of accepting one new sample on every clock cycle, the input ready signal is permanently asserted HIGH, meaning the module never stalls the sender.

```
assign s_axis_fir_tready = 1'b1;
wire in_xfer = s_axis_fir_tvalid && s_axis_fir_tready;
wire out_xfer = m_axis_fir_tvalid && m_axis_fir_tready;
```

The wire 'in_xfer' is HIGH only when a genuine AXI-Stream transfer is occurring on the input side, that is, when valid data is present and the module is ready to accept it. Similarly, 'out_xfer' is HIGH when the downstream module is accepting the filtered output. These two signals are used throughout the module to gate all data movement.

Delay Line (Shift Register)

To compute the filtered output, the filter needs access to the current sample and the previous `NUM_TAPS - 1` samples all at the same time. These are stored in a shift register, which is a chain of registers where each new sample enters at one end and pushes all the older samples one step further down the chain.

```
reg signed [15:0] x[0:NUM_TAPS-1];
integer i;

always @(posedge clk) begin
    if (!reset) begin
        for (i = 0; i < NUM_TAPS; i = i + 1)
            x[i] <= 16'sd0;
        end
    else if (in_xfer) begin
        x[0] <= s_axis_fir_tdata;
        for (i = 1; i < NUM_TAPS; i = i + 1)
            x[i] <= x[i-1];
        end
    end
end
```

On every rising clock edge where ‘in_xfer’ is HIGH, the incoming sample is written into `x[0]` and every existing sample advances one position: `x[1]` receives the old `x[0]`, `x[2]` receives the old `x[1]`, and so on until the oldest sample is displaced from `x[NUM_TAPS-1]` and discarded.

When the active-low reset is asserted (`reset = 0`), all registers are cleared to zero so the filter always starts from a known state. During synthesis, the for loop is fully unrolled into a physical chain of flip-flops on the FPGA fabric.

Filter Coefficients

The filter coefficients are stored in an array `h[0]` through `h[NUM_TAPS-1]` and represented in Q15 fixed-point format, meaning each real-valued floating-point coefficient has been multiplied by $2^{15}=32768$ and rounded to the nearest integer. This representation allows the entire filter computation to be carried out using integer arithmetic on the FPGA, avoiding the need for a floating-point unit.

```
reg signed [15:0] h[0:NUM_TAPS-1];

initial begin
    h[0]  = 16'hFFEF;
    h[1]  = 16'hFF89;
    h[2]  = 16'hFEE9;
    h[3]  = 16'hFF13;
    h[4]  = 16'h01C5;
    h[5]  = 16'h0823;
    h[6]  = 16'h111E;
    h[7]  = 16'h1940;
    h[8]  = 16'h1C8C;
    h[9]  = 16'h1940;
    h[10] = 16'h111E;
    h[11] = 16'h0823;
    h[12] = 16'h01C5;
    h[13] = 16'hFF13;
    h[14] = 16'hFEE9;
    h[15] = 16'hFF89;
    h[16] = 16'hFFEF;
end
```

The coefficients are loaded in an initial block, which executes once at startup. These values never change during the entire operation. Observing the values, the coefficients are symmetric about the center, with the largest values at positions 7 and 8. This symmetry is a defining characteristic of a linear-phase FIR filter. Mathematically, it guarantees that the group delay is constant, which preserves the shape of the ECG waveform as it passes through the filter.

Multiply-Accumulate Operation

The filtering computation itself is a dot product between the delay line ($x[n]$) and the coefficient array ($h[n]$), expressed as:

$$y[n] = h[0] \cdot x[0] + h[1] \cdot x[1] + h[2] \cdot x[2] + \dots + h[N-1] \cdot x[N-1]$$

where $y[n]$ is the filtered output for the current sample, $x[0]$ is the most recent input, and $x[N-1]$ is the oldest sample in the delay line. This is implemented as a combinatorial always block, meaning the result is computed instantly and continuously whenever any sample or coefficient value changes.

```
reg signed [35:0] acc;
integer j;

always @(*) begin
    acc = 36'sd0;
    for (j = 0; j < NUM_TAPS; j = j + 1)
        acc = acc + x[j] * h[j];
end
```

The accumulator (acc) is 36 bits wide, which is a deliberate choice to prevent arithmetic overflow. Each individual multiplication of a 16-bit sample by a 16-bit coefficient produces a result up to 32 bits wide, and summing 16 such products requires an additional $\log_2(16) = 4$ bits of headroom, bringing the minimum safe width to 36 bits. Because both arrays are declared as signed, Verilog automatically performs two's complement signed multiplication, correctly handling negative sample values and negative coefficients without any additional logic. The result stored in acc is in Q30 fixed-point format which is a consequence of multiplying two Q15 numbers together, since the fractional bit count doubles ($15 + 15 = 30$). During synthesis, the for loop is unrolled into a parallel array of hardware multipliers and adders.

Output Register and Format Conversion

Before the result can be transmitted, the Q30 accumulator value must be scaled back to Q15 format to match the scale of the original ECG samples. This is done by arithmetic right-shifting the accumulator by 15 positions, which discards the lower 15 fractional bits introduced by the multiplication step.

```
always @(posedge clk) begin
    if (!reset) begin
        m_axis_fir_tvalid <= 1'b0;
        m_axis_fir_tdata  <= 32'sd0;
        m_axis_fir_tlast  <= 1'b0;
        m_axis_fir_tkeep  <= 4'hF;
    end
    else begin
        if (in_xfer && (!m_axis_fir_tvalid || m_axis_fir_tready)) begin
            m_axis_fir_tdata <= acc >>> 15;
            m_axis_fir_tvalid <= 1'b1;
            m_axis_fir_tlast <= s_axis_fir_tlast;
            m_axis_fir_tkeep <= 4'hF;
        end
        else if (out_xfer) begin
            m_axis_fir_tvalid <= 1'b0;
        end
    end
end
```

The >>> operator performs an arithmetic right shift, which fills the vacated high-order bits with copies of the sign bit rather than with zeros. Using a logical right shift (>>) would corrupt negative values by incorrectly filling those bits with zeros, breaking two's complement representation. The arithmetic shift correctly preserves the sign for both positive and negative filtered results.

The output valid flag `m_axis_fir_tvalid` is asserted HIGH when a new filtered result is available for the downstream module to read. The condition `(!m_axis_fir_tvalid || m_axis_fir_tready)` provides output backpressure protection. The result register is only updated when it is either empty or has just been accepted by the downstream module in the same clock cycle. This prevents a valid result from being silently overwritten before it has been read. Once the downstream module accepts the output, `m_axis_fir_tvalid` is cleared to LOW, resetting the output register to an empty state ready for the next result.

4.4.2 UART Receiver Module

This module receives serial data from the UART interface and converts it into 8-bit parallel data. It generates a valid signal to indicate when a complete byte has been successfully received.

```
1  `timescale 1ns / 1ps
2  module uart_rx (
3      input wire      clk,
4      input wire      reset,    // active high
5      input wire      rx,
6      output reg [7:0] data,
7      output reg      valid
8  );
9      parameter CLK_PER_BIT = 868; // 100MHz / 115200 baud
10
11     localparam IDLE  = 2'd0,
12                 START = 2'd1,
13                 DATA = 2'd2,
14                 STOP  = 2'd3;
15
16     reg [9:0] clk_cnt;
17     reg [2:0] bit_idx;
18     reg [1:0] state;
19     reg [7:0] rx_data;
20
21     always @(posedge clk) begin
22         valid <= 1'b0;
23         if (reset) begin
24             state <= IDLE;
25             clk_cnt <= 0;
26             bit_idx <= 0;
27         end else begin
28             case (state)
```

We have used a FSM to model the receiver module. We collect serial data input bits in the 'rx' variable and make one full byte output in the 'data' variable.

We have 4 states. When there is no valid input bits on the line, we are in IDLE state. When data starts coming, we move to START state. When we are receiving data, we are in DATA state. Finally when data stops, we are in STOP state.

One important parameter is the CLK_PER_BIT. We are using 115200 baud rate. Our FPGA clock is 100MHz. Doing calculation, we find that one bit stays on the line for 868 clock cycles. So our CLK_PER_BIT = 868.

```
28         case (state)
29             IDLE: begin
30                 if (rx == 1'b0) begin    // falling edge = start bit
31                     clk_cnt <= 1;
32                     state <= START;
33                 end
34             end
35             START: begin
36                 if (clk_cnt == CLK_PER_BIT/2) begin
37                     if (rx == 1'b0) begin    // confirm start bit at midpoint
38                         clk_cnt <= 1;
39                         bit_idx <= 0;
40                         state <= DATA;
41                     end else
42                         state <= IDLE;
43                 end else
44                     clk_cnt <= clk_cnt + 1;
45             end
```

In IDLE state, the input line is always HIGH. When we start sending our data, we start with a 0 bit. So, when this module detects a zero bit on the line, it gets activated and moves to the START state. Now the START state counts 868/2 clock cycles to reach to the middle of the bit. If it samples a

zero, the data is coming and we move to the next state. If it is not zero, then it was due to noise or glitch and we move back to IDLE state.

```

46     DATA: begin
47         if (clk_cnt == CLK_PER_BIT) begin
48             rx_data[bit_idx] <= rx;
49             clk_cnt <= 1;
50             if (bit_idx == 3'd7)
51                 state <= STOP;
52             else
53                 bit_idx <= bit_idx + 1;
54             end else
55                 clk_cnt <= clk_cnt + 1;
56         end
57     STOP: begin
58         if (clk_cnt == CLK_PER_BIT) begin
59             valid <= 1'b1;
60             data <= rx_data;
61             state <= IDLE;
62             clk_cnt <= 0;
63         end else
64             clk_cnt <= clk_cnt + 1;
65         end
66     endcase
67 end
68 end
69 endmodule

```

When we have detected that there is a zero on the line, it means the next bits are going to be data bits. So in DATA state, we count 868 clock cycles and sample a bit. Then reset the counter and start again. We do this 8 times for 8 bits. Then we move to the next state.

In the STOP state, we set 'valid = 1'. This signals the 'top' module that our input byte is ready. We load the byte in the 'data' variable and send it to the 'top' module. We then go back to IDLE state and stay there until another zero is detected on the line.

Finite State Diagram of the Receiver module

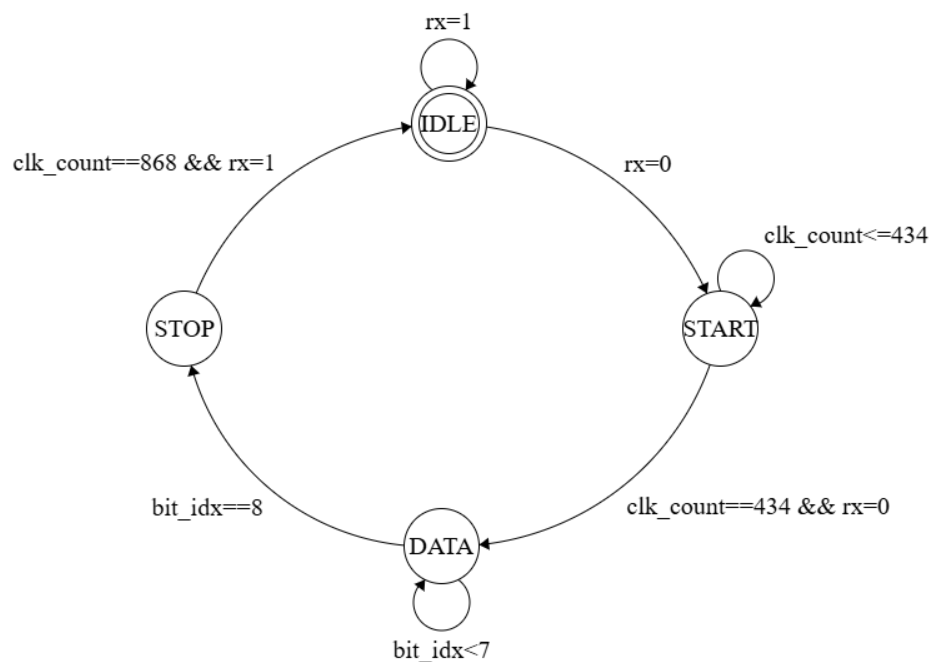


Figure 4: Finite State Diagram of the Receiver module

Baud rate and clock cycles relation

We are using 115200 bauds. So 115200 bits/sec.

$$\text{So, } T_{bit} = \frac{1}{115200} = 8.68\mu s$$

We have a clock of 100MHz

$$\text{So, } T_{clk} = \frac{1}{100 \times 10^6} = 10ns$$

$$\text{So, clocks per bit} = \frac{8.68\mu s}{10ns} = 868$$

We sample at the mid-bit by counting clocks to the middle of the bit for a stable sample.

4.4.3 UART Transmitter Module

The UART transmitter module is responsible for sending the filtered ECG output from the FPGA back to the PC over a serial connection. It takes in a single byte and a start signal, converts the byte into a timed sequence of bits on a single wire, and transmits it at a fixed baud rate that both the FPGA and the PC have agreed upon in advance.

UART is an asynchronous protocol, meaning there is no shared clock signal between the transmitter and receiver. Instead, both sides are configured to the same baud rate, which is 115200 bits per second in this project. A transmission always follows a fixed frame structure: one start bit (logic LOW) to signal the beginning of a byte, eight data bits sent LSB first, and one stop bit (logic HIGH) to signal the end. The receiver on the PC side detects the falling edge of the start bit and uses it to align its own internal timing counter, after which it samples the line at the center of each expected bit period to read the data.

Module Interface and Baud Rate Parameter

The module has a single parameter CLK_PER_BIT which determines the baud rate. It is calculated by dividing the system clock frequency by the desired baud rate:

$$CLK_PER_BIT = F_{clk} / Baud\ rate = 100,000,000 / 115,200 \approx 868$$

This means each transmitted bit is held on the line for exactly 868 clock cycles, which at 100 MHz corresponds to 8.68 microseconds. If a different baud rate or clock frequency were used, only this parameter would need to change.

```
module uart_tx (
    input wire      clk,
    input wire      reset,
    input wire [7:0] data,
    input wire      start,
    output reg      tx,
    output reg      busy
);
    parameter CLK_PER_BIT = 868;
```

Here, 'data' is the 8-bit byte to be transmitted. 'start' is pulsed HIGH for one clock cycle by the top-level state machine to initiate a transmission. 'tx' is the serial output line that carries the actual bit

stream to the PC. 'busy' is driven HIGH by this module for the entire duration of a transmission, signaling to the caller that it must not assert start again until busy returns LOW. This handshake prevents a new byte from being loaded while the current one is still being sent.

State Machine Structure

The transmitter is implemented as a four-state Finite State Machine (FSM). Each state corresponds to a distinct phase of the UART frame transmission, and the machine progresses through them in order for every byte that is sent.

```
localparam IDLE = 2'd0,
           START = 2'd1,
           DATA = 2'd2,
           STOP  = 2'd3;

reg [9:0] clk_cnt;
reg [2:0] bit_idx;
reg [1:0] state;
reg [7:0] tx_data;
```

The four states are IDLE (waiting for a transmission request), START (sending the start bit), DATA (sending the eight data bits one at a time), and STOP (sending the stop bit). 'clk_cnt' is a 10-bit counter that counts clock cycles within each bit period, running from 1 up to 'CLK_PER_BIT' (868) before the state machine moves on. 'bit_idx' is a 3-bit counter that tracks which of the eight data bits is currently being transmitted. 'tx_data' is a local copy of the input data byte, latched at the moment transmission begins so that external changes to data during transmission do not corrupt the outgoing byte.

On reset, the tx line is driven HIGH. This is not arbitrary. The UART protocol defines the idle state of the line as logic HIGH, so if the line were LOW during reset, the receiver on the PC side would misinterpret it as a continuous start bit and produce framing errors.

```
always @(posedge clk) begin
    if (reset) begin
        state    <= IDLE;
        tx       <= 1'b1;
        busy     <= 1'b0;
        clk_cnt  <= 0;
        bit_idx  <= 0;
    end else begin
```

The UART transmitter operates using a finite state machine (FSM) that controls the sequential transmission of data bits over the serial line. The FSM consists of four states: IDLE, START, DATA, and STOP, each responsible for a specific phase of transmission.

1. IDLE State:

In the IDLE state, the transmitter sits passively with 'tx' held HIGH and 'busy' LOW. When the top-level state machine asserts 'start', the module latches the incoming byte into 'tx_data', asserts 'busy' HIGH to block any further transmission requests, initializes the clock counter, and transitions to the START state.

```

IDLE: begin
    tx    <= 1'b1;
    busy  <= 1'b0;
    if (start) begin
        tx_data <= data;
        busy    <= 1'b1;
        clk_cnt <= 1;
        state   <= START;
    end
end

```

Latching ‘data’ into the local ‘tx_data’ register at this moment is important. The top-level module may change data on the very next clock cycle to prepare the next byte, but since the transmitter works from its own internal copy, this has no effect on the byte currently being transmitted.

2. START State:

The start bit is the mechanism by which the receiver synchronizes itself to the incoming data stream. The transmitter pulls ‘tx’ LOW and holds it there for exactly one full bit period. The receiver on the PC detects this falling edge from HIGH to LOW and uses it as the reference point from which it calculates when to sample each of the subsequent data bits.

```

START: begin
    tx <= 1'b0;      // start bit
    if (clk_cnt == CLK_PER_BIT) begin
        clk_cnt <= 1;
        bit_idx <= 0;
        state   <= DATA;
    end else
        clk_cnt <= clk_cnt + 1;
end

```

The clock counter ‘clk_cnt’ increments every cycle. When it reaches ‘CLK_PER_BIT’, exactly one bit period has elapsed, the counter resets to 1, ‘bit_idx’ is initialized to 0, and the state machine moves to DATA to begin transmitting the actual byte content.

3. DATA State:

In the DATA state, each bit of ‘tx_data’ is placed on the ‘tx’ line one at a time and held there for one full bit period before moving to the next. UART transmits the least significant bit first, so ‘tx_data[0]’ is sent first, followed by ‘tx_data[1]’ through ‘tx_data[7]’.

```

DATA: begin
    tx <= tx_data[bit_idx];
    if (clk_cnt == CLK_PER_BIT) begin
        clk_cnt <= 1;
        if (bit_idx == 3'd7)
            state <= STOP;
        else
            bit_idx <= bit_idx + 1;
    end else
        clk_cnt <= clk_cnt + 1;
end

```

On each clock cycle, ‘tx’ is continuously driven with ‘tx_data[bit_idx]’, holding that bit steady on the line. When ‘clk_cnt’ reaches ‘CLK_PER_BIT’, one bit period has passed. If ‘bit_idx’ has reached 7, all eight bits have been sent and the state machine moves to STOP. Otherwise, ‘bit_idx’ increments and the process repeats for the next bit. The receiver on the PC samples the line at the midpoint of each of these 868-cycle windows, reading the stable center of each bit period rather than the edges where transitions could cause errors.

4. STOP State:

After all eight data bits have been transmitted, the transmitter sends a stop bit by driving 'tx' HIGH for one full bit period. The stop bit serves two purposes: it returns the line to its idle state, and it guarantees that the receiver sees a HIGH-to-LOW transition at the start of any subsequent byte, since the next start bit must always be a falling edge. Without the stop bit, back-to-back bytes could run together and the receiver would lose its frame synchronization.

```
STOP: begin
    tx <= 1'b1;    // stop bit
    if (clk_cnt == CLK_PER_BIT) begin
        state <= IDLE;
        busy <= 1'b0;
        clk_cnt <= 0;
    end else
        clk_cnt <= clk_cnt + 1;
    end
end
```

Once the stop bit period is complete, 'busy' is cleared to LOW, signaling to the top-level state machine that the transmitter is free to accept the next byte. The state machine returns to IDLE and the 'tx' line remains HIGH, ready for the next falling edge that will begin the following frame.

Finite State Diagram of the Transmitter module

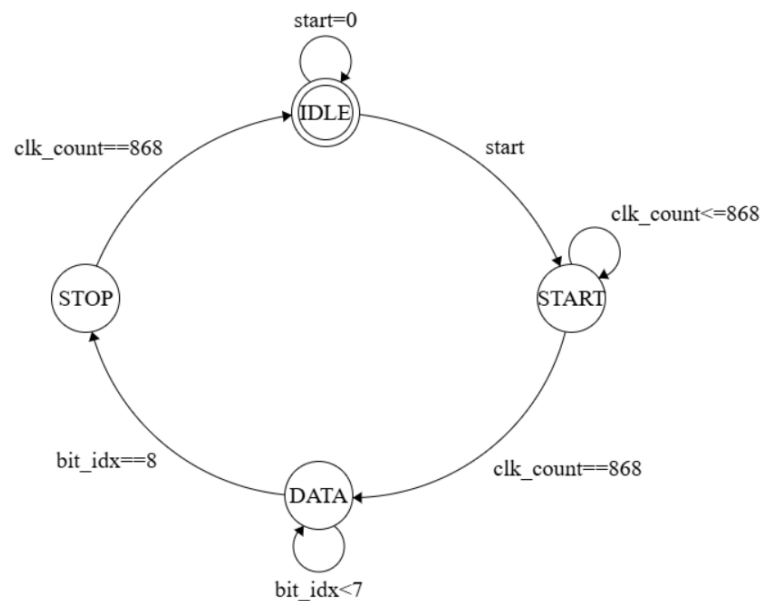


Figure 5: Finite State Diagram of the Transmitter module

4.4.4 Top-Level Module

The top module acts as the central controller of the ECG denoising system, integrating UART communication with the FIR filter core. It manages the complete data flow from input acquisition to filtered output transmission.

The module interfaces with:

- The UART receiver (uart_rx module) for incoming ECG samples
- The FIR filter (FIR module) for signal processing
- The UART transmitter (uart_tx module) for sending filtered results back

Part -1: Module Instantiation

```
1  `timescale 1ns / 1ps
2  module top (
3      input wire clk,
4      input wire reset_btn,
5      input wire uart_rx_pin,
6      output wire uart_tx_pin
7  );
8      wire fir_reset_n = ~reset_btn;
9
10     // -----
11     // UART RX
12     // -----
13     wire [7:0] rx_byte;
14     wire rx_valid;
15
16     uart_rx #(.CLK_PER_BIT(868)) u_rx (
17         .clk (clk),
18         .reset (reset_btn),
19         .rx (uart_rx_pin),
20         .data (rx_byte),
21         .valid (rx_valid)
22     );
23
24     // -----
25     // UART TX
26     // -----
27     reg [7:0] tx_byte;
28     reg tx_start;
29     wire tx_busy;
30
31     uart_tx #(.CLK_PER_BIT(868)) u_tx (
32         .clk (clk),
33         .reset (reset_btn),
34         .data (tx_byte),
35         .start (tx_start),
36         .tx (uart_tx_pin),
37         .busy (tx_busy)
38     );
```

The top module begins by declaring the system inputs and outputs, including the clock, reset signal, and UART communication pins.

The UART receiver (UART RX) is instantiated to accept incoming serial data and convert it into parallel 8-bit data (rx_byte) along with a valid signal (rx_valid). Similarly, the UART transmitter (UART TX) is instantiated to send processed data serially, using control signals such as tx_start and tx_busy to manage transmission.

```

40 // -----
41 // FIR Filter
42 // -----
43 reg signed [15:0] fir_in_data;
44 reg              fir_in_valid;
45 wire signed [31:0] fir_out_data;
46 wire              fir_out_valid;
47 wire              fir_in_ready;
48
49 FIR fir_inst (
50     .clk          (clk),
51     .reset         (fir_reset_n),
52     .s_axis_fir_tdata (fir_in_data),
53     .s_axis_fir_tvalid (fir_in_valid),
54     .s_axis_fir_tlast  (1'b0),
55     .s_axis_fir_tready (fir_in_ready),
56     .m_axis_fir_tdata  (fir_out_data),
57     .m_axis_fir_tvalid (fir_out_valid),
58     .m_axis_fir_tlast  (),
59     .m_axis_fir_tkeep  (),
60     .m_axis_fir_tready (1'b1)
61 );

```

The FIR filter module is instantiated to perform ECG signal denoising. The input sample (fir_in_data) is provided as a signed 16-bit value along with a valid signal (fir_in_valid) to indicate when new data is available. The filter produces a 32-bit signed output (fir_out_data) with an associated valid signal (fir_out_valid).

The module follows an AXI-Stream style interface, where data transfer is controlled using valid and ready signals. In this design, the output is always ready to be accepted, simplifying the data flow. This block represents the core signal processing unit of the system.

Part -2: Finite State Machine Design

```

63 // -----
64 // State machine
65 // -----
66 localparam S_RECV_B0 = 4'd0,
67             S_RECV_B1 = 4'd1,
68             S_FILTER  = 4'd2,
69             S_SEND_B0 = 4'd3,
70             S_WAIT_B0  = 4'd4,
71             S_SEND_B1 = 4'd5,
72             S_WAIT_B1  = 4'd6,
73             S_SEND_B2 = 4'd7,
74             S_WAIT_B2  = 4'd8,
75             S_SEND_B3 = 4'd9,
76             S_WAIT_B3  = 4'd10;
77
78 reg [3:0] state;
79 reg [7:0] byte_lo;
80 reg signed [31:0] result;
81 reg            fir_sent;
82 reg            tx_was_busy;
83
84 always @(posedge clk) begin
85     tx_start    <= 1'b0;
86     fir_in_valid <= 1'b0;
87
88     if (reset_btn) begin
89         state    <= S_RECV_B0;
90         fir_sent  <= 1'b0;
91         tx_was_busy <= 1'b0;
92         fir_in_data <= 16'sd0;
93     end else begin
94         case (state)

```

Next we have built a FSM for operating the whole system. There are receiving, filtering and transmitting states. We receive 2 bytes through the UART port of our FPGA board in serial bit form. Then when the uart_rx module converts them into parallel bytes, we give them as input to the FIR module for filtering. Next we transmit the received output bytes from the FIR module in serial bit form via the uart_tx module. All this is done in states using the FSM.

When the reset signal is asserted, all internal registers and control variables are initialized, and the FSM returns to its initial state (S_RECV_B0). In normal operation, the FSM transitions between states using a case statement, enabling sequential control of data reception, filtering, and transmission.

```

94 case (state)
95
96 // ---- Receive low byte (LSB) of 16-bit sample ----
97 S_RECV_B0: begin
98     if (rx_valid) begin
99         byte_lo <= rx_byte;
100         state <= S_RECV_B1;
101     end
102 end
103
104 // ---- Receive high byte (MSB), assemble sample ----
105 S_RECV_B1: begin
106     if (rx_valid) begin
107         fir_in_data <= $signed({rx_byte, byte_lo});
108         fir_sent <= 1'b0;
109         state <= S_FILTER;
110     end
111 end

```

In the first state, when the uart_rx module has prepared a parallel byte from serial bit data, rx_valid is triggered. We then receive the LSB of 2 bytes and move to the next state.

Next we again wait for the other byte from the uart_rx module. When the next byte is ready, we receive the byte and concatenate the 2 bytes to form the 16 bit sample data.

```

113 // ---- Pulse FIR valid, wait for output ----
114 S_FILTER: begin
115     if (!fir_sent) begin
116         fir_in_valid <= 1'b1;
117         fir_sent <= 1'b1;
118     end
119     if (fir_out_valid) begin
120         result <= fir_out_data;
121         state <= S_SEND_B0;
122     end
123 end

```

Next we move to the filtering state. Here the 16 bit sample is sent to the FIR module for filtering. Now in our design, when we input a current sample, we get the output data from the previous sample. So in this state we give the filter one sample, and receive the filtered data for the previous sample after 1 clock pulse, and move to the next state.

```

125 // ---- Send byte 0 (bits 7:0) ----
126 S_SEND_B0: begin
127     tx_byte <= result[7:0];
128     tx_start <= 1'b1;
129     tx_was_busy <= 1'b0;
130     state <= S_WAIT_B0;
131 end
132 S_WAIT_B0: begin
133     if (tx_busy) tx_was_busy <= 1'b1;
134     if (tx_was_busy && !tx_busy) state <= S_SEND_B1;
135 end
136
137 // ---- Send byte 1 (bits 15:8) ----
138 S_SEND_B1: begin
139     tx_byte <= result[15:8];
140     tx_start <= 1'b1;
141     tx_was_busy <= 1'b0;
142     state <= S_WAIT_B1;
143 end
144 S_WAIT_B1: begin
145     if (tx_busy) tx_was_busy <= 1'b1;
146     if (tx_was_busy && !tx_busy) state <= S_SEND_B2;
147 end

```

On the next state, we have the filtered 4 byte output. We need to send them in serial bits. So first we send the LSB using the uart_tx module. We give tx_start signal and start waiting for the uart_tx module to finish sending the byte. After it is done, we move on to the next state for sending another 8 bits of data. Similarly in this state we trigger the uart_tx module and wait for it to finish sending bits [15:8] and then move to the next state.

```

149 : // ---- Send byte 2 (bits 23:16) ----
150 : S_SEND_B2: begin
151 :     tx_byte    <= result[23:16];
152 :     tx_start    <= 1'b1;
153 :     tx_was_busy <= 1'b0;
154 :     state    <= S_WAIT_B2;
155 : end
156 : S_WAIT_B2: begin
157 :     if (tx_busy)            tx_was_busy <= 1'b1;
158 :     if (tx_was_busy && !tx_busy) state    <= S_SEND_B3;
159 : end
160 :
161 : // ---- Send byte 3 (bits 31:24) ----
162 : S_SEND_B3: begin
163 :     tx_byte    <= result[31:24];
164 :     tx_start    <= 1'b1;
165 :     tx_was_busy <= 1'b0;
166 :     state    <= S_WAIT_B3;
167 : end
168 : S_WAIT_B3: begin
169 :     if (tx_busy)            tx_was_busy <= 1'b1;
170 :     if (tx_was_busy && !tx_busy) state    <= S_RECV_B0;
171 : end
172 :
173 : default: state <= S_RECV_B0;
174 : endcase
175 : end
176 : end
177 :
178 : endmodule

```

On the next state, we send the 8 bits from [23:16]. Just like the previous state, we trigger the uart_tx module and wait until the transmission is finished. Then we move to the next state and send [31:24] and when its done, we have successfully sent our 32 bits of data for one output sample. Now we again move to our receiving LSB state. This is how the whole state machine works.

Thus, this top module integrates all the modules together and performs the filtering operation in states. We receive data, then filter it, then send the data back, and only then we are ready to receive again.

Finite State Diagram of the Top module

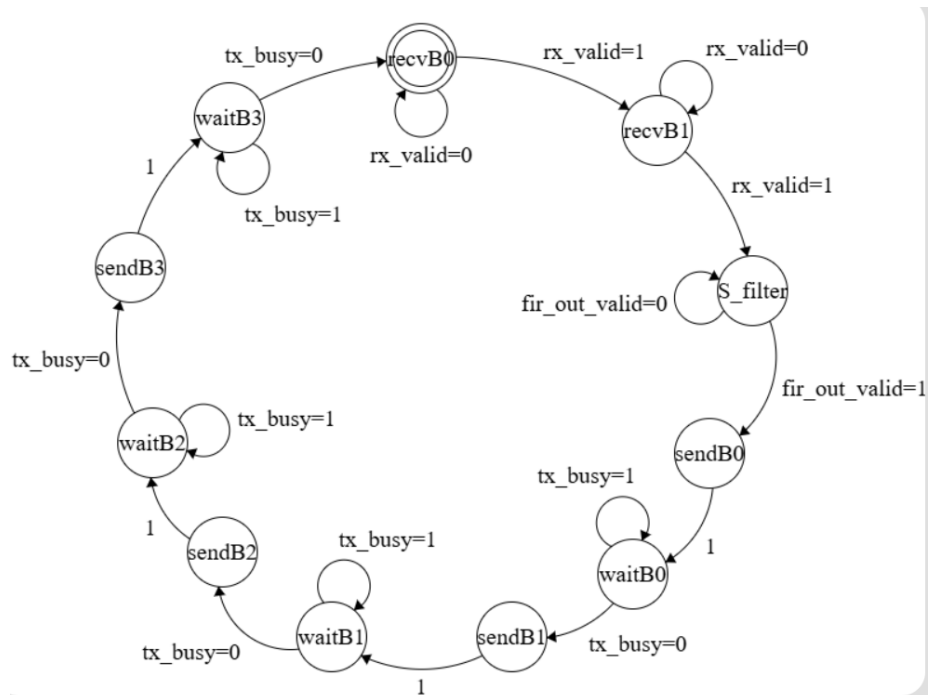


Figure 6: Finite State Diagram of the Top module

4.5 UART Communication and Python Interface

We have used a Python-based interface to communicate between the computer and the FPGA implementing the FIR filter. Communication is established through UART over a USB connection using the PySerial library.

The Python program reads ECG samples from a text file, sends them to the FPGA for processing, and receives the filtered output. The results are then stored in an output file for plotting.

```
import serial
import struct
import time
import sys

# -- Configuration -----
PORT = 'COM4'          # Windows: check Device Manager -> Ports (COM & LPT)
                        # Look for "Silicon Labs CP210x" - that's your Nexys A7
                        # Linux/Mac: usually '/dev/ttyUSB0'
BAUD = 115200
INPUT_FILE = 'ecg.txt'
OUTPUT_FILE = 'ecg_filtered.txt'
# -----

def main():
    # 1. Load samples
    with open(INPUT_FILE, 'r') as f:
        samples = [int(line.strip()) for line in f if line.strip()]
        print("Loaded {} ECG samples from {}".format(len(samples), INPUT_FILE))

    # 2. Clamp to 16-bit signed range (just in case)
    samples_clamped = [max(-32768, min(32767, s)) for s in samples]

    filtered = []
```

We first read our input samples from our input text file, which contains the samples in Q15 format. Then we save those samples in an array named `samples`.

For ensuring bit size correctness, we check whether all values are within 16 bit range. If not, we limit them within this limit.

Then we create our output array and move to the next stage.

```
# 3. Open serial port
try:
    ser = serial.Serial(PORT, BAUD, timeout=5)
    ser = serial.Serial(PORT, BAUD, timeout=10)
except serial.SerialException as e:
    print("ERROR: Cannot open {}".format(PORT))
    print(str(e))
    print("Check that the Nexys A7 is connected and the COM port is correct.")
    sys.exit(1)

print("Connected to {} at {} baud.".format(PORT, BAUD))
time.sleep(1)
ser.reset_input_buffer()
ser.reset_output_buffer()

# 4. Send each sample and collect filtered output
for i, sample in enumerate(samples_clamped):
    # Pack as 16-bit signed, little-endian (2 bytes: LSB first)
    tx_bytes = struct.pack('<h', sample)
    ser.write(tx_bytes)

    # Read 32-bit signed result, little-endian (4 bytes)
    rx_bytes = ser.read(4)
    if len(rx_bytes) < 4:
        print("\nTIMEOUT at sample {}. Only got {} bytes back.".format(i, len(rx_bytes)))
        print("Check that the FPGA is programmed and the board is on.")
        break

    value = struct.unpack('<i', rx_bytes)[0]
    filtered.append(value)
```

Here we establish the connection with the UART using the serial library. We define the PORT that our UART is connected to, the BAUD rate which we selected to be 115200. And we select a timeout for how long we will wait for any given output sample.

After connection is initiated, we wait 1 second for stabilizing the connection. Then clear any garbage data and move to the next stage.

We convert each sample into 2 bytes in little-endian format using `'struct.pack'` command. So the LSB is transported first then the MSB, by the UART chip. After sending a sample, we wait until 4 bytes are received, using the `'ser.read'` command. If timeout happens, it means there was an error. If

not, then our sample is collected in 'rx_bytes'. We convert it into 32 bit signed integer in little-endian format, as the LSB was sent first. Then we append the sample in our 'filtered' array for saving the output.

```
# Progress update every 100 samples
if (i + 1) % 100 == 0 or (i + 1) == len(samples):
    print("Progress: {}/{} samples processed".format(i + 1, len(samples)))

ser.close()
print("\nDone. {} samples received.".format(len(filtered)))

# 5. Write output file
with open(OUTPUT_FILE, 'w') as f:
    for v in filtered:
        f.write("{}\n".format(v))
    print("Filtered signal saved to {}".format(OUTPUT_FILE))

if __name__ == '__main__':
    main()
```

We kept a progress monitoring system so that we could see how many samples were collected, after each 100 samples.

After getting all the samples we close the UART connection via 'ser.close' command.

Finally we write the obtained output sample values into a text file and save it for plotting and comparing with the original signal. This is how our python interface gives input and collects output from the FPGA ,by sending a sample, waiting for the output sample, then moving to the next sample.

5 Design Analysis and Evaluation

5.1 Novelty

The proposed system demonstrates a hardware-based approach to ECG signal denoising by integrating digital signal processing techniques with FPGA implementation. Unlike conventional software-based filtering methods, the use of FPGA enables real-time processing of ECG signals with minimal latency.

The novelty of the design lies in the complete end-to-end implementation, where signal generation, noise modeling, filtering, and visualization are integrated into a unified workflow. The system combines multiple domains, including signal processing, hardware design, and software interfacing, resulting in a robust and efficient ECG denoising solution.

Furthermore, the use of fixed-point arithmetic (Q15 format) and UART-based communication ensures efficient utilization of hardware resources while maintaining acceptable accuracy, making the design suitable for embedded and real-time biomedical applications.

5.2 Design Considerations

The proposed ECG denoising system is designed as a solution to a complex engineering problem involving biomedical signal processing. The design integrates hardware and software components to meet the functional requirements while considering public health, safety, environmental impact, and societal needs. The following subsections outline these considerations.

5.2.1 Considerations to public health and safety

The primary application of the proposed system lies in processing ECG signals, which are critical for diagnosing cardiovascular conditions. Therefore, ensuring the accuracy and integrity of the ECG waveform is essential from a public health perspective.

To address this, a linear-phase FIR filter is employed to avoid phase distortion, ensuring that important features such as the P-wave, QRS complex, and T-wave are preserved. Any distortion in these features could lead to incorrect medical interpretation, potentially affecting patient safety.

Additionally, the system is designed to minimize noise without excessively altering the signal morphology. This ensures that the filtered ECG signal remains reliable for diagnostic purposes.

From a safety standpoint, the FPGA-based implementation ensures deterministic and stable operation, reducing the likelihood of unpredictable system behavior during real-time processing.

5.2.2 Considerations to environment

Environmental considerations are addressed through the efficient use of hardware resources and energy. FPGA-based implementations are generally more power-efficient compared to general-purpose processors when performing dedicated signal processing tasks.

The use of fixed-point arithmetic (Q15 format) further reduces computational complexity and power consumption, as it avoids the overhead associated with floating-point operations.

Moreover, the system design emphasizes compactness and efficiency, which can contribute to the development of portable and energy-efficient biomedical devices. This aligns with sustainable engineering practices by minimizing energy usage and reducing environmental impact.

5.2.3 Considerations to cultural and societal needs

From a societal perspective, access to affordable and efficient healthcare technologies is a major concern, especially in resource-limited settings. The proposed FPGA-based ECG denoising system offers a cost-effective solution by utilizing relatively low-cost hardware while delivering real-time performance.

The system can be adapted for use in portable or remote healthcare monitoring devices, enabling broader accessibility to cardiac diagnostics. This is particularly beneficial in rural or underserved areas where advanced medical infrastructure may not be readily available.

Culturally, the system design supports the growing demand for personalized and home-based healthcare solutions. By enabling reliable ECG signal processing in compact devices, the design aligns with modern healthcare trends that emphasize convenience and accessibility.

5.3 Investigations

5.3.1 Design of Experiment

To evaluate the performance of the ECG denoising system, a controlled experiment was conducted using a simulated ECG signal with additive noise and real-time FPGA-based processing.

A synthetic ECG waveform was generated in Python using Gaussian functions to model the P, Q, R, S, and T wave components. The signal was sampled at 360 Hz to match the FPGA design specifications. Muscle noise was introduced as additive white Gaussian noise to simulate real-world interference.

The noisy ECG signal was converted into 16-bit fixed-point (Q15 format) and transmitted to the FPGA through a UART interface operating at 115200 baud. A Python-based serial communication program was used to send each sample as a 16-bit value and receive the filtered output as a 32-bit value.

The FPGA processed the signal in real time using a FIR low-pass filter, and the filtered output was transmitted back to the computer for analysis and visualization.

To further investigate the effect of filter order on system performance, additional experiments were conducted using simulation with varying numbers of taps.

5.3.2 Data Collection

The dataset consisted of a 5-second ECG signal corresponding to a heart rate of 60 bpm. Muscle noise was added with a controlled amplitude to degrade the signal quality.

The data acquisition process was carried out as follows:

- ECG signal with noise generated in Python
- Signal normalized and converted to Q15 fixed-point format
- Data transmitted serially to FPGA using UART
- FPGA processed each sample using FIR filtering
- Filtered output (32-bit) received via UART
- Output data stored in a file for analysis and plotting

This procedure was used to obtain the hardware-based results for the implemented filter. For additional analysis of different filter orders, simulation-based results were collected using a testbench environment.

5.3.3 Results and Analysis

The performance of the ECG denoising system was evaluated using the input, output, and overlay plots, as shown in Figure.

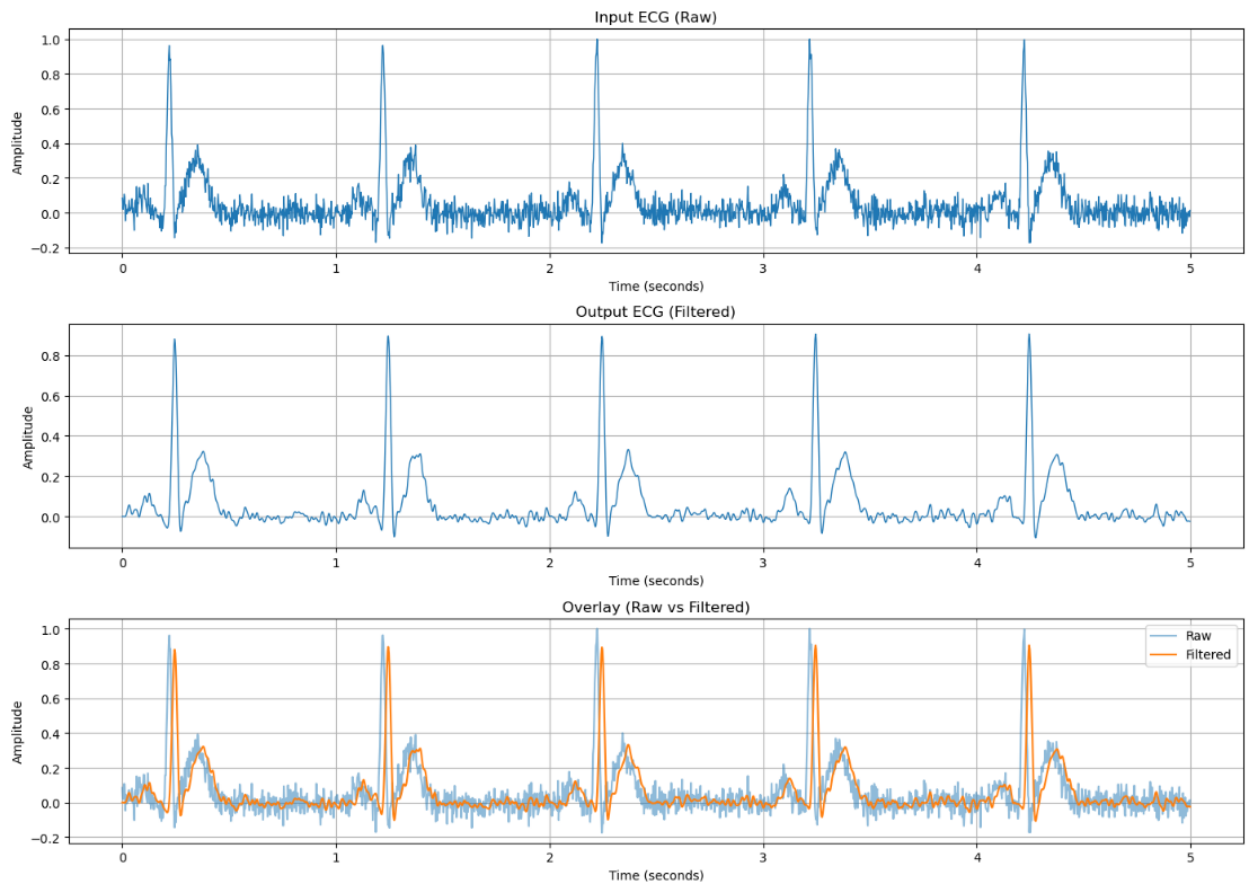


Figure 7: Input ECG (raw), filtered ECG, and overlay comparison

The raw ECG signal contains noticeable high-frequency noise due to simulated muscle artifacts. This noise appears as rapid fluctuations around the baseline, reducing signal clarity. After filtering, the output ECG signal becomes significantly smoother, indicating effective suppression of high-frequency components.

The filtered signal preserves essential ECG features such as the P wave, QRS complex, and T wave. From the overlay plot, it is observed that the filtered signal closely follows the overall shape of the original ECG waveform while removing unwanted noise. The amplitude of critical peaks, particularly the R peaks, remains largely unchanged.

A small delay is observed between the input and output signals due to the inherent group delay of the FIR filter. This delay is consistent and does not distort the waveform morphology.

The primary results correspond to the FIR filter implemented on FPGA hardware with 17 taps. This configuration was selected as a balance between filtering performance and hardware complexity.

To further analyze the effect of filter order, additional experiments were conducted using testbench simulation with varying numbers of taps. It was observed that increasing the number of taps improves noise suppression and produces a smoother ECG signal. However, higher-order filters introduce greater delay and increased computational complexity. In contrast, lower-order filters provide faster response but are less effective in removing noise.

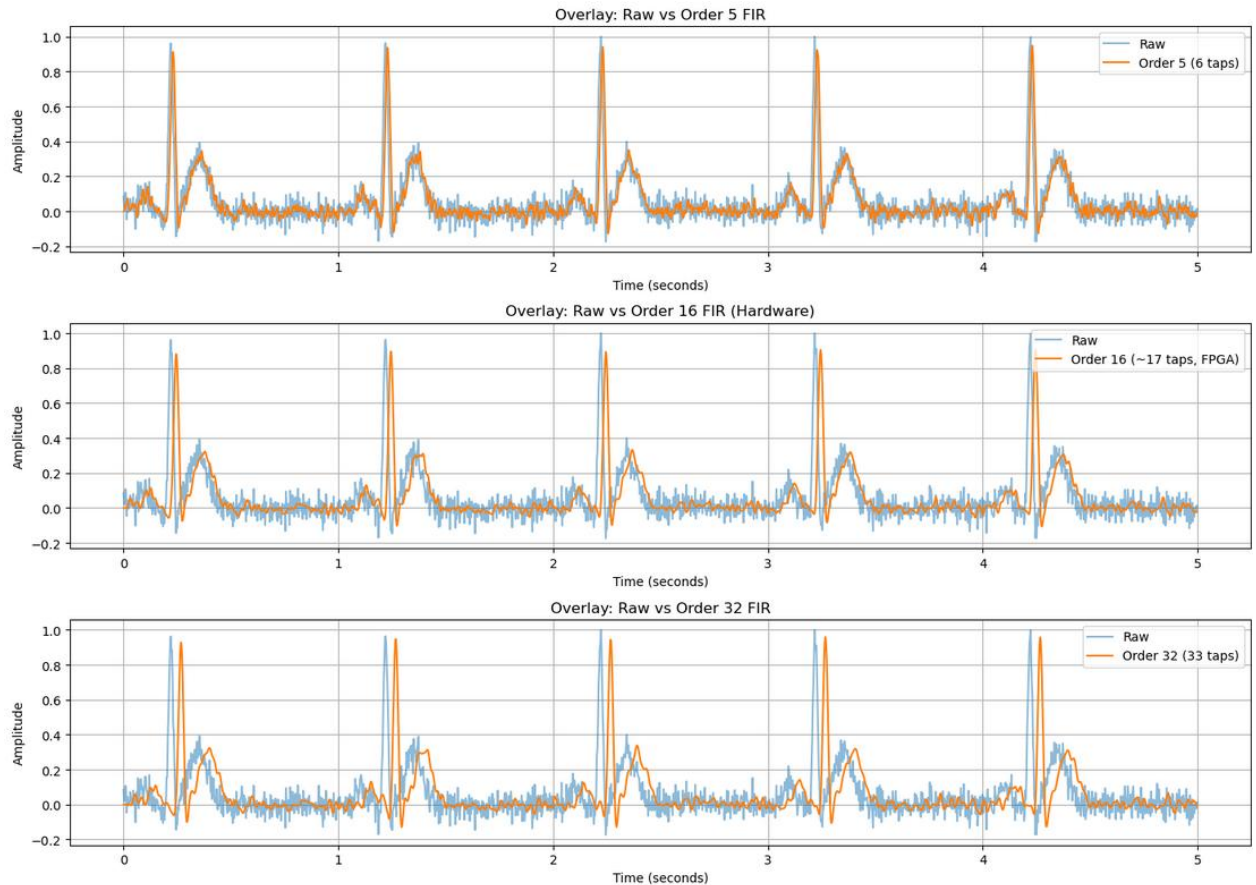


Figure 8: ECG filtering using 5, 16 and 32 order FIR filter

Thus, the results demonstrate that a trade-off exists between filtering performance and system latency. Higher-order filters provide improved noise suppression and smoother signals, while lower-order filters offer reduced delay but less effective noise removal. The choice of filter order therefore depends on the specific requirements of the application.

5.3.4 Interpretation and Conclusions on Data

The experimental results demonstrate that the implemented FIR filter is effective for real-time ECG signal denoising using FPGA hardware.

The system successfully reduces high-frequency noise while preserving the essential structure of the ECG waveform. The consistency between the raw and filtered signals confirms that the filter does not introduce significant distortion.

The investigation of varying filter orders further highlights the trade-off between noise reduction and system complexity. Higher tap counts provide improved filtering performance but at the cost of increased delay and resource usage, whereas lower tap counts offer reduced complexity with less effective noise suppression.

Overall, the proposed system achieves a good balance between performance and efficiency, validating its suitability for real-time ECG signal processing applications.

5.4 Limitations of Tools

The development of the ECG denoising system relies on a combination of software and hardware tools, each of which introduces specific limitations that influence the design process and system performance. Understanding these constraints is essential for making informed engineering decisions and achieving a balanced implementation.

The FIR filter design is carried out using MATLAB's filter design tool, which provides efficient and accurate methods for generating filter coefficients. However, this tool primarily operates at a high level of abstraction and does not account for hardware-level constraints such as finite word length effects, resource utilization, and timing requirements. As a result, additional steps such as fixed-point conversion and manual optimization are required to make the design suitable for FPGA implementation.

The conversion of floating-point coefficients into Q15 fixed-point representation introduces quantization errors, which may slightly alter the frequency response of the filter. This limitation highlights the trade-off between numerical precision and hardware efficiency in digital system design.

Hardware implementation on the Nexys A7 FPGA is subject to resource and timing constraints. The number of available logic elements, multipliers, and memory blocks is limited, which restricts the maximum achievable filter order and overall system complexity. Additionally, timing requirements must be satisfied to ensure correct operation at the desired clock frequency, further constraining the design.

The UART interface used for communication between the host system and the FPGA imposes limitations in terms of data throughput. Due to its relatively low transmission rate, the interface becomes a bottleneck when handling large datasets or higher sampling rates. This restricts the system's ability to fully utilize the high-speed processing capability of the FPGA.

The Python-based communication interface, implemented using serial libraries, is sensitive to configuration parameters such as baud rate and port settings. Any mismatch or instability in the serial connection can result in data loss, communication errors, or increased latency, which affects overall system reliability.

The Verilog hardware description language provides precise control over digital logic implementation; however, debugging at this level can be time-consuming and complex. Identifying and resolving issues related to timing, synchronization, and data flow often requires detailed waveform analysis and iterative testing.

Furthermore, the Vivado simulation environment is primarily used for functional verification of the design. While it accurately validates logical behavior, it does not fully capture real-world hardware effects such as signal propagation delays, noise, and physical implementation variations. Consequently, minor discrepancies may arise between simulation results and actual hardware performance.

Overall, these limitations highlight the practical challenges associated with implementing a real-time ECG denoising system. The design process involves careful trade-offs between accuracy, hardware resource utilization, communication efficiency, and system reliability to achieve an effective and implementable solution.

5.5 Impact Assessment

5.5.1 Assessment of Societal and Cultural Issues

The development of ECG signal processing systems has significant societal implications, particularly in the context of improving access to healthcare technologies. The proposed FPGA-based ECG denoising system demonstrates how engineering solutions can contribute to more efficient and real-time biomedical signal analysis. Such systems can support the development of portable and low-cost diagnostic tools, which may improve healthcare accessibility in resource-limited or remote regions.

From a cultural perspective, the adoption of medical technologies varies across different communities and regions. Factors such as awareness, trust in technology, and availability of skilled personnel influence the acceptance of such systems. Therefore, it is important that these solutions are designed with usability and adaptability in mind, ensuring that they can be integrated into diverse healthcare environments without creating barriers to use.

Additionally, we must consider the societal responsibility of ensuring that technological advancements do not widen the gap between different socio-economic groups. Efforts should be made to design cost-effective and scalable solutions that can benefit a broader population rather than being limited to advanced healthcare facilities.

5.5.2 Assessment of Health and Safety Issues

The application of ECG denoising systems in healthcare directly impacts patient safety and diagnostic reliability. Accurate filtering of ECG signals is essential to preserve critical waveform features such as the P-wave, QRS complex, and T-wave. Any distortion or loss of these features due to improper filtering may lead to incorrect medical interpretation and potential misdiagnosis.

Therefore, ensuring the accuracy and reliability of the system is a key responsibility. The design must be thoroughly validated through both software-level and hardware-level simulations before deployment. Proper testing helps to confirm that the filter effectively removes noise while maintaining the integrity of the original signal.

In addition, real-time processing systems must operate consistently without data loss or communication errors. Issues such as UART transmission errors or synchronization problems could affect the quality of the processed signal. We must ensure that robust design practices and verification procedures are followed to minimize such risks.

Overall, safety considerations require that the system performs reliably under all expected operating conditions and that its limitations are clearly understood before being used in any clinical context.

5.5.3 Assessment of Legal Issues

The implementation of biomedical signal processing systems involves certain legal considerations, particularly when such systems are intended for healthcare applications. Medical devices and related technologies are typically subject to regulatory standards and approval processes to ensure their safety and effectiveness.

Although the proposed system is developed as an academic project, any transition to real-world deployment would require compliance with relevant medical device regulations and standards. This includes validation, documentation, and certification processes to ensure that the system meets established guidelines for accuracy, reliability, and safety.

Another important legal aspect is data handling. ECG signals represent sensitive patient information, and proper measures must be taken to ensure data privacy and security. We must ensure that appropriate safeguards are in place to prevent unauthorized access, data loss, or misuse of patient data.

Furthermore, we have a professional responsibility to accurately represent the capabilities and limitations of the system. Misrepresentation or overstatement of performance could lead to improper use and potential liability issues.

In summary, legal considerations emphasize the importance of regulatory compliance, data protection, and responsible engineering practices in the development and application of biomedical systems.

5.6 Sustainability Evaluation

The sustainability of the proposed ECG denoising system is evaluated by considering its environmental, economic, social, and technical impacts, along with the associated professional responsibilities. As the system is intended for real-time biomedical signal processing, it is important to assess both its immediate benefits and long-term implications.

From an environmental perspective, the use of an FPGA-based implementation offers improved energy efficiency compared to general-purpose computing systems for dedicated signal processing tasks. The system consumes relatively low power during operation, making it suitable for continuous use in real-time applications. Additionally, the digital nature of the design reduces reliance on bulky analog components, thereby minimizing material usage. However, it is also important to consider that electronic hardware, including FPGA boards, contributes to electronic waste at the end of its lifecycle, and proper disposal and recycling practices must be followed.

In terms of economic sustainability, the system benefits from the flexibility and reusability of programmable hardware. Once implemented, the FPGA can be reconfigured for different applications, reducing the need for new hardware and lowering long-term costs. The system also eliminates the need for high-performance computing resources for real-time processing, making it cost-effective for practical deployment. However, the initial development process requires specialized tools and technical expertise, which may increase upfront costs.

From a societal standpoint, the system contributes to improving healthcare technology by enabling efficient and real-time ECG signal processing. This can support the development of portable diagnostic devices, potentially increasing access to healthcare services in remote or resource-limited areas. By enhancing the quality of ECG signals, the system can assist in more accurate diagnosis and better patient outcomes. At the same time, we must ensure that such technologies are accessible to a wide range of users and do not contribute to disparities in healthcare availability.

The system also demonstrates technical sustainability through its modular and scalable design. The use of hardware description language and standard communication interfaces allows for future modifications and upgrades without requiring a complete redesign. Furthermore, the system can be adapted for other signal processing applications, increasing its long-term usability. Nevertheless,

limitations such as finite precision effects and hardware resource constraints must be carefully managed to ensure consistent performance over time.

Finally, ethical and professional responsibility plays an important role in ensuring sustainability. We must ensure that the system is designed and implemented with accuracy, reliability, and transparency. Clearly communicating the system's capabilities and limitations is essential to prevent misuse, particularly in critical healthcare applications.

Overall, the project reflects a sustainable engineering approach by balancing performance, cost, energy efficiency, and societal impact, while maintaining responsibility toward environmental and ethical considerations.

5.7 Ethical Issues

The development of the ECG denoising system involves several ethical considerations, particularly due to its application in biomedical signal processing. As the system deals with health-related data and potential clinical use, it is important to ensure responsible design, implementation, and usage.

One of the primary ethical concerns is the accuracy and reliability of the system. Incorrect filtering of ECG signals may distort important waveform features, leading to misleading interpretations. Such inaccuracies could affect medical decision-making if the system is used without proper validation. Therefore, we must ensure that the filtering process preserves critical signal characteristics and performs consistently under different conditions.

Patient safety is another crucial aspect. The proposed system is developed as an academic and experimental implementation and is not intended for direct clinical diagnosis. Using the system in real medical scenarios without rigorous testing, validation, and certification could pose risks to patients. We must ensure that the system is clearly presented with its limitations and is not misused in safety-critical applications.

Data privacy and security also play an important role. ECG signals represent sensitive personal health information, and improper handling of such data may lead to privacy violations. We must ensure that all data is handled securely, stored responsibly, and protected from unauthorized access during transmission and processing.

Responsible use of technology is an essential ethical requirement in engineering practice. Biomedical systems must undergo extensive testing and validation before being deployed in real-world environments. We must ensure that the system is used only within its intended scope and that any limitations are clearly communicated to users.

Finally, accessibility and fairness must be considered. The development of low-cost, FPGA-based healthcare solutions has the potential to improve access to medical technology. However, we must ensure that such advancements benefit a wide range of users and do not create inequality in access to healthcare resources.

Overall, ethical considerations emphasize the importance of accuracy, safety, data protection, responsible deployment, and fairness. Addressing these factors is essential to ensure that the system contributes positively to society while maintaining professional engineering standards.

6 Reflection on Individual and Team work

6.1 Individual Contribution of Each Member

Student ID	Name	Accomplished Tasks
2106003	Rejoun Rahi Rad	ECG data preparation, noisy ECG handling, Q15 coefficient conversion, Verilog implementation of uart_tx, report writing
2106005	Abrar Fahim	Verilog implementation of uart_rx.v, Filter Designing in MATLAB, FIR simulation and verification, presentation slides making
2106011	Md. Yeasir Alif	Verilog implementation of FIR.v, baud-rate timing logic, PySerial Library handling, UART communication testing, report writing
2106012	Al Muktadir	Verilog implementation of top.v, system-level FSM design, 32-bit output byte framing,, result plotting using Matplotlib, report writing

6.2 Log Book of Project Implementation

Date	Activity / Progress
December 23	The initial project idea was discussed and finalized within the PR group.
January 14	The FIR module was designed in Verilog HDL and its functionality was verified through simulation using a testbench.
February 4	The FIR filter was tested for the first time using the generated ECG signal to evaluate its filtering performance.
February 17	Preliminary FPGA implementation attempts were carried out; however, successful hardware realization was not achieved at this stage.
February 18	Further implementation and debugging were performed to resolve hardware integration and execution issues.
February 19	Additional FPGA implementation and debugging efforts were continued to improve system functionality and stability.
February 20	The complete project was successfully implemented on the FPGA platform.

7 Communication to External Stakeholders

7.1 Executive Summary

This project presents a real-time ECG signal denoising system implemented using an FPGA platform. ECG signals are often affected by noise, which can make analysis difficult and reduce diagnostic accuracy. To address this, a digital FIR low-pass filter is designed and implemented on a Nexys A7 FPGA board. The system receives noisy ECG data from a computer, processes it using the filter, and sends back a cleaner signal. The design combines MATLAB-based filter development, Verilog hardware implementation, and Python-based communication. The results demonstrate effective noise reduction, making the system suitable for efficient and reliable biomedical signal processing applications.

7.2 User Manual

The following steps describe how to operate the ECG denoising system:

Step 1: Prepare Input Signal

- Ensure the ECG signal is stored in a .txt file.
- Convert the signal into **Q15 (16-bit signed integer)** format.
- Each line should contain one sample value.

Step 2: Filter Design and Coefficient Preparation (MATLAB)

- Open the **Filter Designer Tool** in MATLAB.
- Select **FIR Low-Pass Filter**.
- Set the design parameters.
- Generate the filter and export the coefficients.
- Convert the coefficients into **Q15 fixed-point hexadecimal format**.

Step 2: Program the FPGA

- Open the project in Vivado.
- Open the FIR module in Vivado and:
 - Update the parameter NUM_TAPS to the desired tap number.
 - Replace the coefficient values in the `initial` block with the generated hex values.
- Save and re-synthesize the design before programming the FPGA.
- Generate the bitstream.
- Program the Nexys A7 FPGA board.

Step 3: Connect Hardware

- Connect the FPGA board to the computer using a **UART (USB) cable**.
- Identify the correct COM port from the system.

Step 4: Run Python Script

- Update the COM port in the Python code.
- Run the script.
- The script will:
 - Send ECG samples to the FPGA
 - Receive filtered data
 - Save output to a new `.txt` file

Step 5: View Results

- Open the output file.
- Use the provided plotting script to visualize:
 - Raw ECG signal
 - Filtered ECG signal

Step 6: Verification

- Compare raw and filtered signals to confirm noise reduction.

7.3 Github Link

[AFN2760/Fir_Filter_Implementation_FPGA.git](https://github.com/AFN2760/Fir_Filter_Implementation_FPGA.git)

7.4 YouTube Link

<https://www.youtube.com/watch?v=NpFRRqWyVo0>

8 Future Work

While the proposed ECG denoising system demonstrates effective performance, several improvements and extensions can be considered to enhance its capability, efficiency, and practical applicability.

One potential area of improvement is the optimization of the FIR filter architecture for better timing performance on the FPGA. The current implementation uses a straightforward multiply-accumulate structure, which may limit the maximum achievable clock frequency. Future work can focus on optimizing the design to reduce critical path delay and improve overall processing speed.

Another important enhancement involves the use of pipelining or parallel processing techniques. By introducing pipelined stages or parallel computation of filter operations, the system can achieve higher throughput and support real-time processing of higher sampling rate signals. This would significantly improve the performance of the system in demanding applications.

The communication interface can also be improved. The current system uses UART, which has limited data transfer speed and may become a bottleneck in real-time applications. Replacing UART with a faster interface, such as SPI, USB, or Ethernet-based communication, can enable higher data rates and more efficient data transfer between the host system and FPGA.

In addition, the system can be extended to support more advanced filtering techniques. Implementing higher-order filters or adaptive filtering methods would allow the system to handle a wider range of noise conditions and improve signal quality under varying environments.

Currently, the system operates using pre-recorded ECG data from text files. Future development can involve integrating real-time ECG acquisition hardware, enabling the system to process live biomedical signals. This would make the system more suitable for practical healthcare applications.

Finally, the design can be further developed into a portable, low-cost biomedical monitoring system. By combining the FPGA-based processing unit with compact hardware and efficient power management, the system can be adapted for wearable or remote health monitoring applications, increasing its real-world usability and societal impact.

9 References

[1] Jayashree S., Shanthapreetha S., and Velan C., “FPGA Implementation of Optimized FIR Filter for ECG Denoising,” Journal of Theoretical and Applied Information Technology, 2022.

Link: <https://www.ja-tit.org/volumes/Vol100No15/6Vol100No15.pdf>

[2] “FPGA Based FIR Filter Design for Enhancement of ECG Signal,” 2013.

Link: <https://inpressco.com/wp-content/uploads/2013/11/Paper311775-1778.pdf>

[3] Chessda Uttraphan et al., “Hardware Implementation of FIR Filter for ECG Signal Processing: Design, Optimization, and Performance Analysis on an FPGA,” 2024.

Link:

https://www.researchgate.net/publication/378552809_Hardware_Implementation_of_FIR_Filter_for_ECG_Signal_Processing_Design_Optimization_and_Performance_Analysis_on_an_FPGA

[4] Rahul Sharma, Rajesh Mehra, and Chandni, “FPGA based Asynchronous FIR Filter Design for ECG Signal Processing,” International Journal of Computer Applications, 2016.

Link: <https://www.ijcaonline.org/archives/volume156/number7/sharma-2016-ijca-912463.pdf>

[5] M. I. A. Ardani et al., “Hardware Implementation of an FIR Filter for ECG Signal Processing,” 2023.

Link: <https://penerbit.uthm.edu.my/periodicals/index.php/eeee/article/download/10802/3362>

[6] MathWorks, “Getting Started with Filter Designer.”

Link: <https://www.mathworks.com/help/signal/ug/getting-started-with-filter-designer.html>